music

April 1, 2025

1 Set Up

[198]:
```
# Import
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.ensemble import IsolationForest
from sklearn.svm import SVC  # Support Vector Classification
import lightgbm as lgb
from imblearn.ensemble import BalancedRandomForestClassifier
from sklearn.metrics import classification_report , f1_score, recall_score,␣
↪confusion_matrix, ConfusionMatrixDisplay
from imblearn.over_sampling import SMOTE
from sklearn.impute import SimpleImputer
```

[199]:
```
# Functions
def most_frequent(series):
    return series.mode()[0] if not series.mode().empty else None
```

[200]:
```
# Settings

pd.set_option('display.max_columns', None)  # Show all columns
pd.set_option('display.width', 1000)  # Prevent line breaks
pd.set_option('display.max_colwidth', None)  # Show full column values
```

[204]:
```
# Read Files

# Path to your Parquet file
parquet_file = '/Users/henryding/Documents/Seminar/Data/output_data1.parquet'

# Read the Parquet file into a DataFrame
```

```
df = pd.read_parquet(parquet_file, engine='pyarrow')  # You can also use␣
 ↪'fastparquet'

# Display the first few rows of the DataFrame
display(df.head(1000))
num_rows = df.shape[0]
print(f"The number of rows in the DataFrame is: {num_rows}")
```

```
    status gender      length  firstName level  lastName  registration   userId␣
 ↪           ts       auth         page  sessionId                            ␣
 ↪ location  itemInSession                                                     ␣
 ↪                                                                            ␣
 ↪userAgent                                                                   ␣
 ↪                                                                      song  ␣
 ↪                                                               artist method
0       200      M   524.32934      Shlok  paid   Johnson  1.533735e+12  1749042␣
 ↪ 1538352001000  Logged In     NextSong      22683        Dallas-Fort␣
 ↪Worth-Arlington, TX            278                          "Mozilla/5.0␣
 ↪(Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.
 ↪1985.143 Safari/537.36"                                                     ␣
 ↪                                          Ich mache einen Spiegel -␣
 ↪Dream Part 4                                                                ␣
 ↪Popol Vuh    PUT
1       200      F   238.39302    Vianney  paid    Miller  1.537500e+12  1563081␣
 ↪ 1538352002000  Logged In     NextSong      20836      San␣
 ↪Francisco-Oakland-Hayward, CA               9                 "Mozilla/5.0␣
 ↪(Macintosh; Intel Mac OS X 10_9_4) AppleWebKit/537.36 (KHTML, like Gecko)␣
 ↪Chrome/36.0.1985.143 Safari/537.36"                                         ␣
 ↪                                                                            ␣
 ↪        MiÃ Â©ntele                                                         ␣
 ↪ Los Bunkers    PUT
2       200      F   140.35546       Vina  paid    Bailey  1.536415e+12  1697168␣
 ↪ 1538352002000  Logged In     NextSong       4593                          ␣
 ↪ Hilo, HI            109                                                    ␣
 ↪   Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:31.0) Gecko/20100101␣
 ↪Firefox/31.0                                                                ␣
 ↪                                                            Baby Talk       ␣
 ↪                                                          Lush    PUT
3       200      M   277.15873     Andres  paid     Foley  1.534387e+12  1222580␣
 ↪ 1538352003000  Logged In     NextSong       6370                          ␣
 ↪Watertown, SD             71                    "Mozilla/5.0 (Macintosh; Intel␣
 ↪Mac OS X 10_9_4) AppleWebKit/537.77.4 (KHTML, like Gecko) Version/7.0.5 Safari/
 ↪537.77.4"                                                                   ␣
 ↪     Horn Concerto No. 4 in E flat K495: II. Romance (Andante cantabile) ␣
 ↪Barry Tuckwell/Academy of St Martin-in-the-Fields/Sir Neville Marriner    PUT
```

```
4       200       F  1121.25342    Aaliyah  paid   Ramirez  1.537381e+12  1714398␣
↪ 1538352003000  Logged In     NextSong      22316          ␣
↪Baltimore-Columbia-Towson, MD             21                         ␣
↪"Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.36 (KHTML, like Gecko)␣
↪Chrome/36.0.1985.143 Safari/537.36"  Close To The Edge (I. The Solid Time Of␣
↪Change_ II. Total Mass Retain_ III. I Get Up I Get Down_ IV. Seasons Of Man)␣
↪(Remastered LP Version)                                                    ␣
↪             Yes    PUT
..      ...    ...         ...        ...    ...        ...          ...       ...            ...   ␣
↪    ...          ...        ...                                    ...            ...   ␣
↪                                                                          ␣
↪                                                     ...                   ␣
↪                                                                          ␣
↪                                      ...                                  ␣
↪                             ...    ...
995     307       F         NaN     Alivia  paid  Williams  1.535955e+12  1792538␣
↪ 1538352525000  Logged In    Thumbs Up       8871  New York-Newark-Jersey␣
↪City, NY-NJ-PA             75                                            ␣
↪        Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:31.0) Gecko/20100101␣
↪Firefox/31.0                                                              ␣
↪                                                                  None    ␣
↪                                                           None    PUT
996     200       M   315.48036  Christian  free     Klein  1.536940e+12  1291813␣
↪ 1538352526000  Logged In     NextSong      14821  New York-Newark-Jersey␣
↪City, NY-NJ-PA             10                      "Mozilla/5.0 (Macintosh;␣
↪Intel Mac OS X 10_9_2) AppleWebKit/537.74.9 (KHTML, like Gecko) Version/7.0.2␣
↪Safari/537.74.9"                                                          ␣
↪                                                                      ␣
↪Odessa                                                            Caribou␣
↪  PUT
997     200       M   248.73751  Kristofer  free     James  1.536879e+12  1929921␣
↪ 1538352526000  Logged In     NextSong       9831            ␣
↪Seattle-Tacoma-Bellevue, WA             56                                ␣
↪                          Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:31.0)␣
↪Gecko/20100101 Firefox/31.0                                               ␣
↪                                                            Jigsaw␣
↪Falling Into Place                                                        ␣
↪ Radiohead    PUT
998     200       F    53.08036       Zoey  free   Gregory  1.533190e+12  1839740␣
↪ 1538352527000  Logged In     NextSong      24848            ␣
↪Phoenix-Mesa-Scottsdale, AZ             11  "Mozilla/5.0 (X11; Linux x86_64)␣
↪AppleWebKit/537.36 (KHTML, like Gecko) Ubuntu Chromium/34.0.1847.116 Chrome/34.
↪0.1847.116 Safari/537.36"                                                 ␣
↪                                                   Broken Hearted Hoover Fixer␣
↪Sucker Guy                                                        Glen␣
↪Hansard    PUT
```

```
999     200      F         NaN       Zoey  free   Gregory  1.533190e+12  1839740␣
↪ 1538352527000  Logged In  Roll Advert      24848          ␣
↪Phoenix-Mesa-Scottsdale, AZ             12  "Mozilla/5.0 (X11; Linux x86_64)␣
↪AppleWebKit/537.36 (KHTML, like Gecko) Ubuntu Chromium/34.0.1847.116 Chrome/34.
↪0.1847.116 Safari/537.36"                                                    ␣
↪                                                                             ␣
↪     None                                                                   ␣
↪None    GET

[1000 rows x 18 columns]
```

```
The number of rows in the DataFrame is: 26259199
```

2 Exploratory Analysis

[205]:
```
# Categorical Variables

unique_level = df['level'].unique()
print(f"level: {unique_level}")

unique_auth = df['auth'].unique()
print(f"auth: {unique_auth}")

unique_method = df['method'].unique()
print(f"method: {unique_method}")

unique_page= df['page'].unique()
print(f"page: {unique_page}")
```

```
level: ['paid' 'free']
auth: ['Logged In' 'Logged Out' 'Cancelled' 'Guest']
method: ['PUT' 'GET']
page: ['NextSong' 'Home' 'Thumbs Up' 'Error' 'Help' 'Add to Playlist'
 'Roll Advert' 'Login' 'Thumbs Down' 'Settings' 'About' 'Add Friend'
 'Upgrade' 'Submit Upgrade' 'Logout' 'Downgrade' 'Save Settings'
 'Submit Downgrade' 'Cancel' 'Cancellation Confirmation' 'Register'
 'Submit Registration']
```

[206]:
```
# User Usage
user_counts = df.groupby('userId').size().reset_index(name='count')
display(user_counts.sort_values(by='count', ascending=False))
```

```
       userId   count
5936   1261737  778479
12534  1564221   13591
20802  1931933   12831
163    1006695   12372
7562   1336969   11858
```

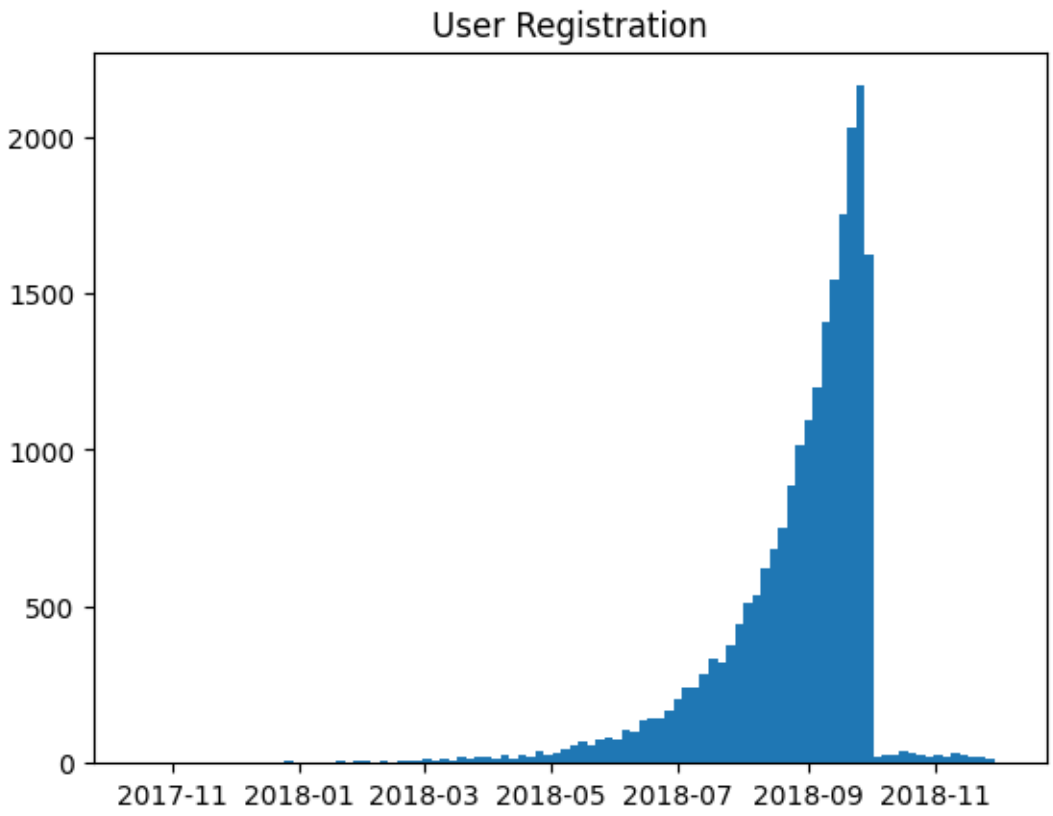
```
…         …      …
6072    1267517       1
9698    1434698       1
17778   1793623       1
9129    1408726       1
15322   1689121       1

[22278 rows x 2 columns]
```

[207]:
```
# Time Period
# Convert Timestamps to datetime
df['ts'] = pd.to_datetime(df['ts'], unit='ms')

print(f"timestamp min: {df['ts'].min()}")
print(f"timestamp max: {df['ts'].max()}")

# Convert Registration to datetime
df['registration'] = pd.to_datetime(df['registration'], unit='ms')

print(f"registration min: {df['registration'].min()}")
print(f"registration max: {df['registration'].max()}")
```

```
timestamp min: 2018-10-01 00:00:01
timestamp max: 2018-12-01 00:00:02
registration min: 2017-10-14 22:05:25
registration max: 2018-12-03 07:23:42
```

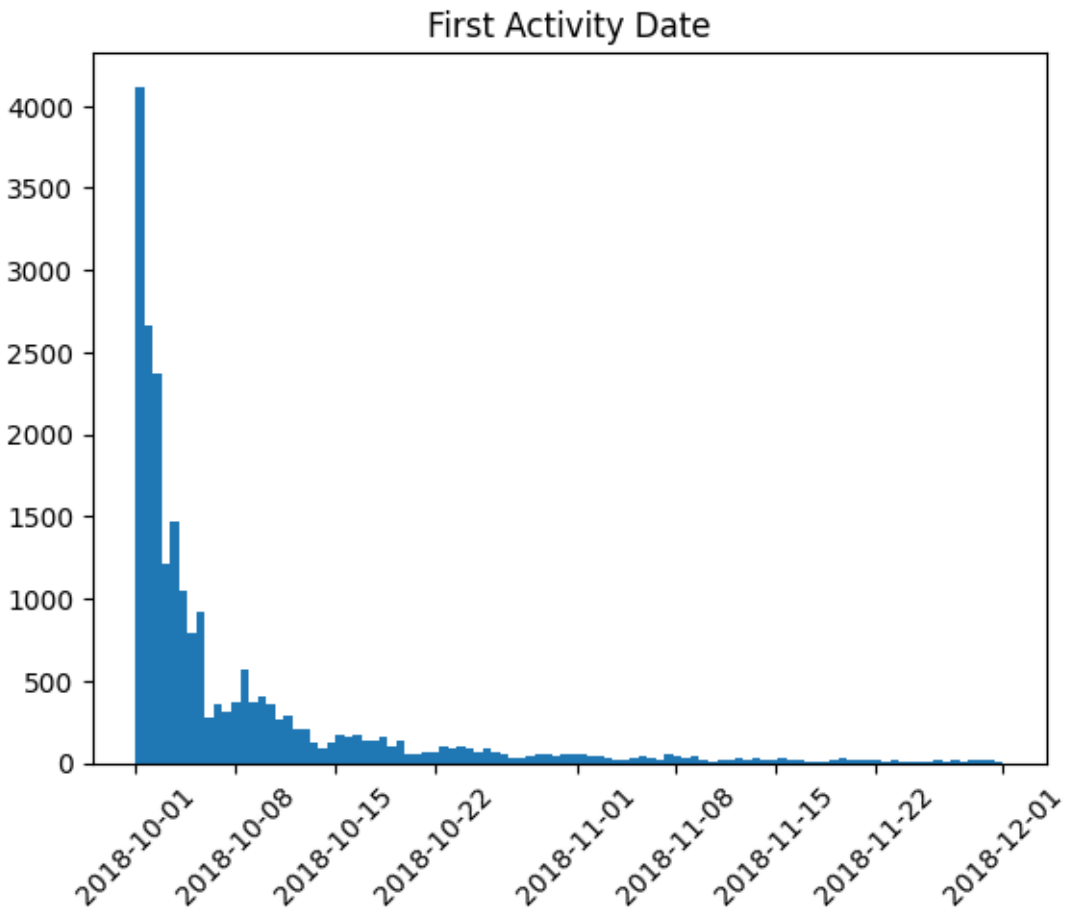
[208]:
```
# User Registration
grouped_registration = df.groupby('userId')['registration'].agg('min')
display(grouped_registration.head())
plt.hist(grouped_registration, bins=100)
plt.title('User Registration')
plt.show()
```

```
userId
1000025   2018-07-10 09:30:08
1000035   2018-09-12 19:28:22
1000083   2018-09-07 18:01:49
1000103   2018-09-22 07:27:25
1000164   2018-08-12 09:32:01
Name: registration, dtype: datetime64[ns]
```



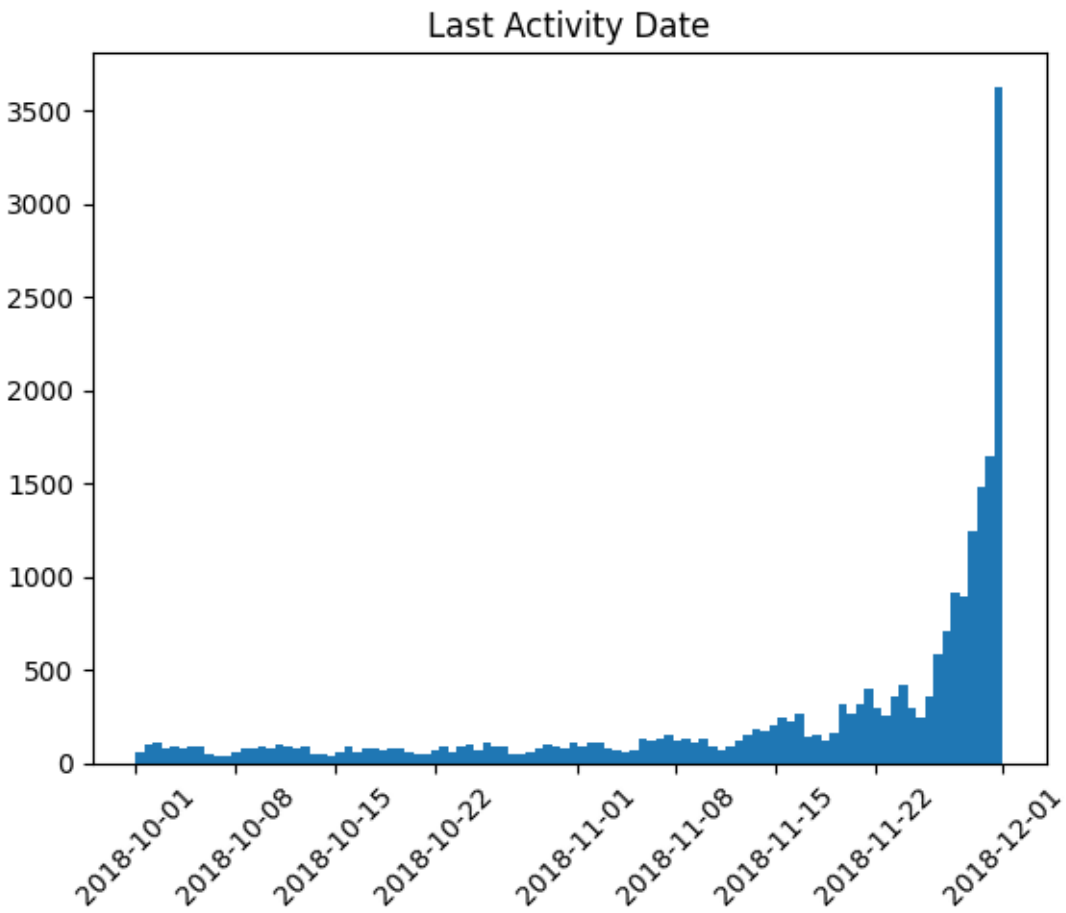
[11]:
```
# First Activity Date
grouped_start = df.groupby('userId')['ts'].agg('min')
display(grouped_start.head())
plt.hist(grouped_start, bins=100)
plt.xticks(rotation=45)
plt.title('First Activity Date')
plt.show()
```

```
userId
1000025   2018-10-02 08:59:29
1000035   2018-10-05 18:29:46
1000083   2018-10-01 05:37:45
1000103   2018-10-04 17:31:17
1000164   2018-10-01 17:40:18
Name: ts, dtype: datetime64[ns]
```



[12]:
```
# Last Activity Date
grouped_end = df.groupby('userId')['ts'].agg('max')
display(grouped_end.head())
plt.hist(grouped_end, bins=100)
plt.xticks(rotation=45)
plt.title('Last Activity Date')
plt.show()
```

```
userId
1000025   2018-10-18 20:33:05
1000035   2018-11-20 09:14:13
1000083   2018-10-12 10:04:58
1000103   2018-11-21 03:01:43
1000164   2018-11-30 16:53:06
Name: ts, dtype: datetime64[ns]
```

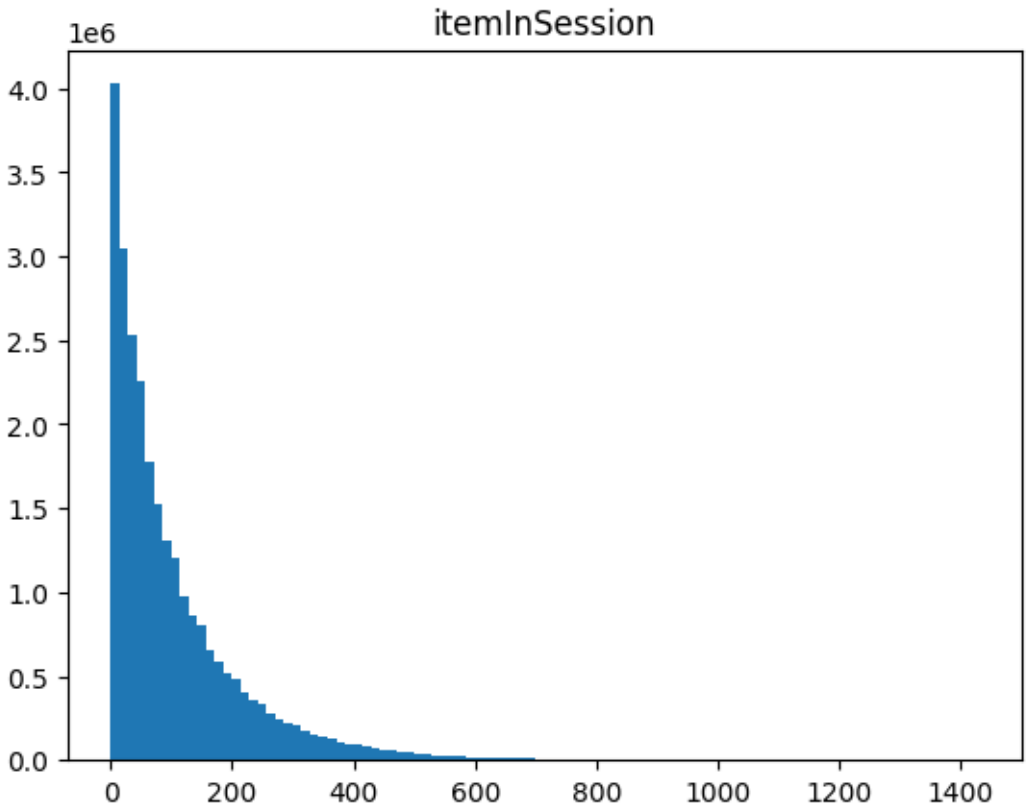


[101]:
```
# High-Cardinality Variables
song_count = df.groupby('song').size().count()
print(f"Number of Songs: {song_count}")

artist_count = df.groupby('artist').size().count()
print(f"Number of Artists: {artist_count}")

user_count = df.groupby('userId').size().count()
print(f"Number of Users: {user_count}")
```

```
Number of Songs: 253564
Number of Artists: 38337
Number of Users: 22278
```

[13]:
```
# Continuous Variables

plt.hist(df['itemInSession'], bins=100)
plt.title("itemInSession")
```

```
plt.show()
```



3 Data Preprocessing

[ ]:

4 Feature Engineering

[209]:
```
# Create "week" feature to group by week
df['week'] = df['ts'].dt.to_period('W').astype('datetime64[ns]')

# Sort by userId and timestamp to calculate session_diff properly
df = df.sort_values(by=['userId', 'ts'])

# Calculate the time difference between consecutive sessions (session duration)
df['session_diff'] = df.groupby('userId')['ts'].diff().fillna(pd.
 ↪Timedelta(seconds=0))

# Calculate the total session duration (in seconds) for each session
```

```
df['session_duration'] = df['session_diff'].dt.total_seconds()
```

[210]:
```
# Dummy Coding

# Page
for click_type in unique_page:
    df[click_type] = np.where(df['page'] == click_type, 1, 0)

# Levels
df['dummy_level'] = np.where(df['level'] == 'paid', 1, 0)
```

4.0.1 Explanation of Features:

- **week**: A new feature to group data by weeks, with each week starting on Monday.
- **session_diff**: Time difference between consecutive sessions, which helps us understand user activity over time.
- **session_duration**: Total session duration in seconds, derived from the session_diff.
- **session_count**: Number of sessions a user has in total.
- **avg_item_in_session**: Average number of items per session.
- **dummy_level**: A binary feature representing whether the user is on a free or paid plan.

[10]:
```
display(df.head())
```

```
       status gender     length firstName level lastName        registration  ␣
↪userId                  ts       auth      page  sessionId                  ␣
↪location  itemInSession                                                      ␣
↪                                                 userAgent          song     ␣
↪    artist method       week    session_diff  session_duration  session_count ␣
↪Home  NextSong  Thumbs Up  Add to Playlist  Roll Advert  Help  Thumbs Down ␣
↪Upgrade  Submit Upgrade  Downgrade  Logout  Add Friend  Settings  About  Save␣
↪Settings  Error  Cancel  Cancellation Confirmation  Submit Downgrade  Login ␣
↪Register  Submit Registration  dummy_level
403570     200      M        NaN    Camren  free   Walker 2018-07-10 09:30:08 ␣
↪1000025 2018-10-02 08:59:29  Logged In      Home      23706  New␣
↪Haven-Milford, CT              0  "Mozilla/5.0 (Windows NT 6.1; WOW64)␣
↪AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.1985.143 Safari/537.36"    ␣
↪     None              None    GET 2018-10-01 0 days 00:00:00               0.
↪0     122.931172     1         0          0                0            0    ␣
↪0            0        0               0          0       0           0       ␣
↪0      0              0      0       0                          0           ␣
↪   0      0         0                   0           0
```

```
404235     200      M  251.97669    Camren  free   Walker 2018-07-10 09:30:08 ␣
 ↪1000025 2018-10-02 09:02:44  Logged In  NextSong      23706  New␣
 ↪Haven-Milford, CT              1  "Mozilla/5.0 (Windows NT 6.1; WOW64)␣
 ↪AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.1985.143 Safari/537.36"  ␣
 ↪Underground              1001    PUT 2018-10-01 0 days 00:03:15            ␣
 ↪195.0     122.931172     0         1          0                0           0 ␣
 ↪   0           0        0               0          0       0           0     ␣
 ↪   0      0             0      0       0                          0         ␣
 ↪      0      0         0                    0            0
405134     200      M  223.05914    Camren  free   Walker 2018-07-10 09:30:08 ␣
 ↪1000025 2018-10-02 09:06:55  Logged In  NextSong      23706  New␣
 ↪Haven-Milford, CT              2  "Mozilla/5.0 (Windows NT 6.1; WOW64)␣
 ↪AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.1985.143 Safari/537.36" ␣
 ↪AsesÃ Âname  Charly GarcÃ Ãa    PUT 2018-10-01 0 days 00:04:11             251.
 ↪0     122.931172     0         1          0                0           0    ␣
 ↪0           0        0               0          0       0           0       ␣
 ↪0      0             0      0       0                          0            ␣
 ↪   0      0         0                    0            0
405929     200      M  193.38404    Camren  free   Walker 2018-07-10 09:30:08 ␣
 ↪1000025 2018-10-02 09:10:38  Logged In  NextSong      23706  New␣
 ↪Haven-Milford, CT              3  "Mozilla/5.0 (Windows NT 6.1; WOW64)␣
 ↪AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.1985.143 Safari/537.36"  ␣
 ↪Last Nite       The Strokes    PUT 2018-10-01 0 days 00:03:43             223.
 ↪0     122.931172     0         1          0                0           0    ␣
 ↪0           0        0               0          0       0           0       ␣
 ↪0      0             0      0       0                          0            ␣
 ↪   0      0         0                    0            0
406597     200      M  177.37098    Camren  free   Walker 2018-07-10 09:30:08 ␣
 ↪1000025 2018-10-02 09:13:51  Logged In  NextSong      23706  New␣
 ↪Haven-Milford, CT              4  "Mozilla/5.0 (Windows NT 6.1; WOW64)␣
 ↪AppleWebKit/537.36 (KHTML, like Gecko) Chrome/36.0.1985.143 Safari/537.36"  ␣
 ↪  The End         Pearl Jam    PUT 2018-10-01 0 days 00:03:13             193.
 ↪0     122.931172     0         1          0                0           0    ␣
 ↪0           0        0               0          0       0           0       ␣
 ↪0      0             0      0       0                          0            ␣
 ↪   0      0         0                    0            0
```

5 Defining Churn and Activity

[240]:
```
# Identify users who have data for the current week (check for missing rows for␣
 ↪each user)
all_users = df['userId'].unique()
weeks_in_df = df['week'].unique()

# Create a DataFrame of all possible combinations of userId and week (even for␣
 ↪missing users)
```

```
all_combinations = pd.MultiIndex.from_product([all_users, weeks_in_df],␣
 ↪names=['userId', 'week'])
df_all_weeks = pd.DataFrame(index=all_combinations).reset_index()

# Create a DataFrame that groups by UserId and Week with the features you are␣
 ↪interested in
df_actuals = df.groupby(['userId', 'week']).agg(
    session_count = ('itemInSession', 'mean'),
    session_duration = ('session_duration', 'sum'),
    dummy_level = ('dummy_level', 'first')
)
# Merge with the original data to identify users missing for a given week
df_all_weeks = df_all_weeks.merge(df_actuals, on=['userId', 'week'], how='left')

# Sort by userId and week to handle previous week properly
df_all_weeks = df_all_weeks.sort_values(by=['userId', 'week'])


# Create "prev_week_activity" to track if the user was active in the previous␣
 ↪week
df_all_weeks['prev_week_activity'] = df_all_weeks.groupby('userId',␣
 ↪group_keys=False)['session_count'].shift(1).fillna(0)
df_all_weeks['prev_week_session_duration'] = df_all_weeks.groupby('userId',␣
 ↪group_keys=False)['session_duration'].shift(1).fillna(0)
df_all_weeks['prev_dummy_level'] = df_all_weeks.groupby('userId',␣
 ↪group_keys=False)['dummy_level'].shift(1).fillna(method='ffill').fillna(0)

# Fill NaN session_count, prev_week_activity, and session_duration with 0
df_all_weeks['session_count'] = df_all_weeks['session_count'].fillna(0)
df_all_weeks['session_duration'] = df_all_weeks['session_duration'].fillna(0)

# Fill dummy_level NaN with forward-fill (ffill)
df_all_weeks['dummy_level'] = df_all_weeks['dummy_level'].fillna(method='ffill')

# Define churn: users who were active in the previous week but are inactive now
df_all_weeks['churn'] = np.where(
    ((df_all_weeks['session_count'] == 0) & (df_all_weeks['prev_week_activity']␣
 ↪> 0)),
    1,
    0
)
```

```
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/1790987056.py:2
5: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a
future version. Use obj.ffill() or obj.bfill() instead.
  df_all_weeks['prev_dummy_level'] = df_all_weeks.groupby('userId',
group_keys=False)['dummy_level'].shift(1).fillna(method='ffill').fillna(0)
```

```
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/1790987056.py:3
2: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a
future version. Use obj.ffill() or obj.bfill() instead.
  df_all_weeks['dummy_level'] =
df_all_weeks['dummy_level'].fillna(method='ffill')
```

[ ]:
```python
# Sort for right calculations
df = df.sort_values(by=['ts'], ascending=True)

# Group by Days
df['day'] = df['ts'].dt.date

df_all_days = df.groupby(['userId', 'week', 'day']).agg(
    session_count=('itemInSession', 'mean'),
    session_duration=('session_duration', 'sum'),
    dummy_level=('dummy_level', 'first')
)

# Ensure daily data is sorted
df_all_days = df_all_days.sort_values(by=['userId', 'day'])

# Define rolling window sizes in days
window_sizes = [7, 14, 21, 28]  # Rolling windows of 1, 2, 3, and 4 weeks

# Apply rolling difference calculations
for window in window_sizes:
    df_all_days[f'rolling_diff_{window}_days'] = (
        df_all_days.groupby('userId')['session_count']
        .rolling(window=window, min_periods=1)
        .sum()
        .diff()
        .reset_index(level=0, drop=True)
    )


# Aggregate daily rolling differences to weekly level (take last value in each␣
 ↪week)
df_weekly_rolling_diffs = df_all_days.groupby(['userId', 'week'])[[
    f'rolling_diff_{window}_days' for window in window_sizes
]].last().reset_index()


# Merge corrected rolling diffs back into df_all_weeks
df_all_weeks = df_all_weeks.merge(df_weekly_rolling_diffs, on=['userId',␣
 ↪'week'], how='left')
```

```python
# Sort before shifting
df_all_weeks = df_all_weeks.sort_values(by=['userId', 'week'])

# Apply shift for previous week rolling diffs
for window in window_sizes:
    df_all_weeks[f'prev_week_rolling_diff_{window}_days'] = (
        df_all_weeks.groupby('userId',␣
 ↪group_keys=False)[f'rolling_diff_{window}_days']
        .shift(1)  # Shift values down by one week per user
        .fillna(0)
    )

# # Display final dataset

print(df_all_weeks)
```

```
        userId       week  session_count  session_duration  dummy_level
prev_week_activity  prev_week_session_duration  prev_dummy_level  churn
rolling_diff_7_days  rolling_diff_14_days  rolling_diff_21_days
rolling_diff_28_days  prev_week_rolling_diff_7_days
prev_week_rolling_diff_14_days  prev_week_rolling_diff_21_days
prev_week_rolling_diff_28_days
0       1000025 2018-10-01     171.821705          482224.0          0.0
0.000000                          0.0              0.0      0
68.703704             68.703704             68.703704             68.703704
0.000000                          0.000000                          0.000000
0.000000
1       1000025 2018-10-08      66.317416          350667.0          1.0
171.821705                    482224.0               0.0      0
-27.569620             94.000000             94.000000             94.000000
68.703704                         68.703704                         68.703704
68.703704
2       1000025 2018-10-15      90.768097          591125.0          1.0
66.317416                     350667.0               1.0      0
120.773684            130.273684            130.273684            130.273684
-27.569620                         94.000000                         94.000000
94.000000
3       1000025 2018-10-22       0.000000               0.0          1.0
90.768097                     591125.0               1.0      1
NaN                   NaN                   NaN                   NaN
120.773684                        130.273684                        130.273684
130.273684
4       1000025 2018-10-29       0.000000               0.0          1.0
0.000000                          0.0              1.0      0
NaN                   NaN                   NaN                   NaN
0.000000                          0.000000                          0.000000
0.000000
…          …         …             …                 …          …
```

```
…                          …                …    …                   …
…                   …                    …                             …
…                          …                              …
200497  1999996 2018-10-29      15.950820          199341.0          0.0
39.618799                  1143249.0              0.0      0
-50.707407             17.500000            17.500000            17.500000
20.461538                       24.461538                      24.461538
24.461538
200498  1999996 2018-11-05      25.627451          726498.0          0.0
15.950820                   199341.0              0.0      0
21.627451             25.627451            25.627451            25.627451
-50.707407                       17.500000                      17.500000
17.500000
200499  1999996 2018-11-12       3.500000          455443.0          0.0
25.627451                   726498.0              0.0      0
-25.836207             -6.000000             3.500000             3.500000
21.627451                       25.627451                      25.627451
25.627451
200500  1999996 2018-11-19       0.000000               0.0          0.0
3.500000                   455443.0              0.0      1
NaN                   NaN                   NaN                   NaN
-25.836207                       -6.000000                      3.500000
3.500000
200501  1999996 2018-11-26       6.000000         1225167.0          0.0
0.000000                        0.0              0.0      0
-12.555556              5.500000             6.000000             6.000000
0.000000                        0.000000                      0.000000
0.000000

[200502 rows x 17 columns]
```

[192]:
```
display(df_all_weeks)
display(df_actuals)
```

```
         userId       week  session_count  session_duration  dummy_level ␣
 ↪prev_week_activity  prev_week_session_duration  prev_dummy_level  churn ␣
 ↪rolling_diff_1_weeks  rolling_diff_2_weeks  rolling_diff_3_weeks ␣
 ↪rolling_diff_4_weeks  rolling_diff_5_weeks  rolling_diff_6_weeks ␣
 ↪rolling_diff_7_weeks  rolling_diff_8_weeks
127487  1638340 2018-10-15       0.000000               0.0          0.0        ␣
 ↪   8.000000                    682077.0               0.0      1        ␣
 ↪7.500000              8.000000              8.000000              8.000000    ␣
 ↪           8.000000              8.000000              8.000000              8.
 ↪000000
```

```
123483  1617370 2018-10-22        0.000000                  0.0          0.0         ␣
↪  36.746269                       27440.0                  0.0       1            ␣
↪36.746269           36.746269             36.746269             36.746269  ␣
↪           36.746269            36.746269             36.746269             36.
↪746269
179224  1892578 2018-11-19        0.000000                  0.0          0.0         ␣
↪  27.710145                      586791.0                  0.0       1            ␣
↪-4.833973           27.710145             24.210145              0.539690  ␣
↪           27.710145            27.710145             27.710145             27.
↪710145
72079   1358034 2018-11-19        0.000000                  0.0          1.0         ␣
↪  70.799270                      650214.0                  1.0       1            ␣
↪19.985417          -86.630217             43.864964             61.694007  ␣
↪           16.018782            47.799270             70.799270             70.
↪799270
49255   1239501 2018-11-19        0.000000                  0.0          0.0         ␣
↪  11.000000                      370161.0                  0.0       1            ␣
↪10.000000           11.000000             11.000000             11.000000  ␣
↪           11.000000            11.000000             11.000000             11.
↪000000
…          …        …            …               …           …               …  ␣
↪                    …                …    …                  …                 ␣
↪     …                   …                   …                   …             ␣
↪     …                   …                   …
72040   1357698 2018-10-29       59.946121          617104.0          1.0         ␣
↪  72.725333                      592484.0                  1.0       0          ␣
↪-21.913300           66.689619             55.581103             72.725333  ␣
↪            72.725333            72.725333             72.725333             72.
↪725333
72041   1357698 2018-11-05      164.284264          604744.0          1.0         ␣
↪  59.946121                      617104.0                  1.0       0          ␣
↪-12.779213          -34.692513             53.910406             42.801890  ␣
↪            59.946121            59.946121             59.946121             59.
↪946121
72042   1357698 2018-11-12       99.775126          586588.0          0.0         ␣
↪ 164.284264                      604744.0                  1.0       0          ␣
↪104.338143           91.558931             69.645631            158.248550  ␣
↪           147.140033           164.284264            164.284264            164.
↪284264
72043   1357698 2018-11-19      195.478462          622826.0          1.0         ␣
↪  99.775126                      586588.0                  0.0       0          ␣
↪-64.509138           39.829005             27.049792              5.136492  ␣
↪            93.739411            82.630895             99.775126             99.
↪775126
```

```
200501  1999996 2018-11-26       6.000000         1225167.0          0.0
  ↪   0.000000                          0.0               0.0      0
  ↪-3.500000           -25.627451           -15.950820           -39.618799
  ↪         -4.000000            -3.500000            -8.600000              0.
  ↪000000

[200502 rows x 17 columns]
```

userId	week	session_count	session_duration	dummy_level
1000025	2018-10-01	171.821705	482224.0	0
	2018-10-08	66.317416	350667.0	1
	2018-10-15	90.768097	591125.0	1
1000035	2018-10-01	4.500000	58000.0	0
	2018-10-08	34.097561	459822.0	0
…		…	…	…
1999996	2018-10-22	39.618799	1143249.0	0
	2018-10-29	15.950820	199341.0	0
	2018-11-05	25.627451	726498.0	0
	2018-11-12	3.500000	455443.0	0
	2018-11-26	6.000000	1225167.0	0

```
[121443 rows x 3 columns]
```

[242]:
```python
# Additional Feature Engineering

# Ensure data is sorted by userId and week before applying rolling operations
df_all_weeks = df_all_weeks.sort_values(by=['userId', 'week'])

# Define rolling window sizes
window_sizes = [1, 2, 3, 4, 5, 6, 7, 8]

# Apply rolling difference calculations for multiple window sizes
for window in window_sizes:
    df_all_weeks[f'rolling_diff_{window}_weeks'] = (
        df_all_weeks.groupby('userId')['prev_week_activity']
        .rolling(window=window, min_periods=1)
        .sum()
        .diff()
        .reset_index(level=0, drop=True)  # Ensures proper alignment after␣
 ↪groupby
    )
    # Replace NaN values with 0
    df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
```

```
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
```

```
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.
```

```
  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
/var/folders/fl/6yh7c335749bvjp9pb0rj8fw0000gn/T/ipykernel_20340/4092923539.py:1
9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
```

'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.

```
  df_all_weeks[f'rolling_diff_{window}_weeks'].fillna(0, inplace=True)
```

[243]:
```
df_all_weeks = df_all_weeks.sort_values(by='churn', ascending=False)
display(df_all_weeks.head(100))

print(df_all_weeks['churn'].dtypes)
print(df_all_weeks['churn'].unique())

# Check for NaN.
for column in df_all_weeks.columns:
    nan_count = df_all_weeks[column].isna().sum()
    print(f"Column '{column}' has {nan_count} NaN values.")
```

```
        userId       week  session_count  session_duration  dummy_level ␣
↪prev_week_activity  prev_week_session_duration  prev_dummy_level  churn ␣
↪rolling_diff_7_days  rolling_diff_14_days  rolling_diff_21_days ␣
↪rolling_diff_28_days  prev_week_rolling_diff_7_days ␣
↪prev_week_rolling_diff_14_days  prev_week_rolling_diff_21_days ␣
↪prev_week_rolling_diff_28_days  rolling_diff_1_weeks  rolling_diff_2_weeks ␣
↪rolling_diff_3_weeks  rolling_diff_4_weeks  rolling_diff_5_weeks ␣
↪rolling_diff_6_weeks  rolling_diff_7_weeks  rolling_diff_8_weeks
127487  1638340 2018-10-15            0.0               0.0          0.0       ␣
↪   8.000000                    682077.0               0.0      1             ␣
↪   NaN                    NaN                    NaN                    NaN      ␣
↪                8.000000                       8.000000                      ␣
↪ 8.000000                          8.000000              7.500000             8.
↪000000              8.000000              8.000000              8.000000      ␣
↪       8.000000              8.000000              8.000000
123483  1617370 2018-10-22            0.0               0.0          0.0       ␣
↪  36.746269                    27440.0               0.0      1             ␣
↪   NaN                    NaN                    NaN                    NaN      ␣
↪               67.000000                      67.000000                     ␣
↪67.000000                          67.000000             36.746269            36.
↪746269             36.746269             36.746269             36.746269      ␣
↪      36.746269             36.746269             36.746269
179224  1892578 2018-11-19            0.0               0.0          0.0       ␣
↪  27.710145                    586791.0               0.0      1             ␣
↪   NaN                    NaN                    NaN                    NaN      ␣
↪               53.500000                      53.500000                     ␣
↪53.500000                          53.500000             -4.833973            27.
↪710145             24.210145             0.539690              27.710145      ␣
↪      27.710145             27.710145             27.710145
```

```
72079   1358034 2018-11-19            0.0                 0.0           1.0         ␣
 ↪  70.799270                   650214.0                1.0       1                ␣
 ↪   NaN                    NaN                    NaN                    NaN      ␣
 ↪                24.889189                     94.389189                          ␣
↪94.389189                    94.389189            19.985417                -86.
↪630217               43.864964              61.694007              16.018782      ␣
 ↪      47.799270              70.799270              70.799270
49255   1239501 2018-11-19            0.0                 0.0           0.0         ␣
 ↪  11.000000                   370161.0                0.0       1                ␣
 ↪   NaN                    NaN                    NaN                    NaN      ␣
 ↪                11.000000                     11.000000                          ␣
↪11.000000                    11.000000            10.000000                 11.
↪000000               11.000000              11.000000              11.000000      ␣
 ↪      11.000000              11.000000              11.000000
…          …         …            …              …          …                …  ␣
 ↪                        …                …    …                 …                ␣
 ↪   …                   …                    …                    …               ␣
 ↪                       …                             …                            ␣
 ↪    …                   …                    …                   …               ␣
 ↪     …                   …                    …                   …              ␣
 ↪      …
179338  1893052 2018-10-29            0.0                 0.0           0.0         ␣
 ↪  62.479730                   767589.0                0.0       1                ␣
 ↪   NaN                    NaN                    NaN                    NaN      ␣
 ↪                 9.000000                      9.000000                          ␣
 ↪ 9.000000                     9.000000            62.479730                56.
↪979730               40.592633              62.479730              62.479730      ␣
 ↪      62.479730              62.479730              62.479730
24706   1122985 2018-10-08            0.0                 0.0           1.0         ␣
 ↪  55.347368                    57989.0                1.0       1                ␣
 ↪   NaN                    NaN                    NaN                    NaN      ␣
 ↪                98.014286                     98.014286                          ␣
↪98.014286                    98.014286            55.347368                 55.
↪347368               55.347368              55.347368              55.347368      ␣
 ↪      55.347368              55.347368              55.347368
89956   1448255 2018-10-08            0.0                 0.0           0.0         ␣
 ↪   8.000000                     4013.0                0.0       1                ␣
 ↪   NaN                    NaN                    NaN                    NaN      ␣
 ↪              -280.536042                   -280.536042                        ␣
↪-280.536042                  -280.536042             8.000000                     ␣
↪8.000000              8.000000              8.000000              8.000000        ␣
 ↪         8.000000              8.000000              8.000000
```

```
24746   1123156 2018-11-05            0.0               0.0          1.0
  ↪  68.583799                    687512.0               1.0      1
  ↪   NaN                    NaN                   NaN                    NaN
  ↪               22.000000                        -9.828205
 ↪44.500000                        44.500000            22.509541            31.
 ↪188276             -5.431276           -21.982509             68.583799
  ↪      68.583799             68.583799             68.583799
123438  1617154 2018-10-22            0.0               0.0          1.0
  ↪   7.000000                    69378.0               1.0      1
  ↪   NaN                    NaN                   NaN                    NaN
  ↪                7.000000                         7.000000
  ↪ 7.000000                         7.000000           -53.488688            -0.
 ↪500000              7.000000              7.000000              7.000000
  ↪       7.000000              7.000000              7.000000

[100 rows x 25 columns]
```

```
int64
[1 0]
Column 'userId' has 0 NaN values.
Column 'week' has 0 NaN values.
Column 'session_count' has 0 NaN values.
Column 'session_duration' has 0 NaN values.
Column 'dummy_level' has 0 NaN values.
Column 'prev_week_activity' has 0 NaN values.
Column 'prev_week_session_duration' has 0 NaN values.
Column 'prev_dummy_level' has 0 NaN values.
Column 'churn' has 0 NaN values.
Column 'rolling_diff_7_days' has 79059 NaN values.
Column 'rolling_diff_14_days' has 79059 NaN values.
Column 'rolling_diff_21_days' has 79059 NaN values.
Column 'rolling_diff_28_days' has 79059 NaN values.
Column 'prev_week_rolling_diff_7_days' has 0 NaN values.
Column 'prev_week_rolling_diff_14_days' has 0 NaN values.
Column 'prev_week_rolling_diff_21_days' has 0 NaN values.
Column 'prev_week_rolling_diff_28_days' has 0 NaN values.
Column 'rolling_diff_1_weeks' has 0 NaN values.
Column 'rolling_diff_2_weeks' has 0 NaN values.
Column 'rolling_diff_3_weeks' has 0 NaN values.
Column 'rolling_diff_4_weeks' has 0 NaN values.
Column 'rolling_diff_5_weeks' has 0 NaN values.
Column 'rolling_diff_6_weeks' has 0 NaN values.
Column 'rolling_diff_7_weeks' has 0 NaN values.
Column 'rolling_diff_8_weeks' has 0 NaN values.
```

[148]:
```
display(df_all_weeks[df_all_weeks['userId'] ==          1358034].
 ↪sort_values('week').head(100))
```

```
      userId       week  session_count  session_duration  dummy_level ␣
↪prev_week_activity  prev_week_session_duration  prev_dummy_level  churn ␣
↪rolling_diff_3_weeks  rolling_diff_5_weeks  rolling_diff_10_weeks ␣
↪rolling_diff_15_weeks  rolling_diff_20_weeks  rolling_diff_1_weeks ␣
↪rolling_diff_2_weeks  rolling_diff_4_weeks  rolling_diff_6_weeks ␣
↪rolling_diff_7_weeks  rolling_diff_8_weeks
72072  1358034 2018-10-01      23.000000           10847.0          0.0         ␣
↪  0.000000                          0.0               1.0      0          -188.
↪940993          -188.940993           -231.940993           -231.940993     ␣
↪       -231.940993           -79.438710          -188.940993          -188.
↪940993          -209.940993          -209.940993          -231.940993
72073  1358034 2018-10-08      54.780488          358279.0          0.0         ␣
↪ 23.000000                     10847.0               0.0      0            23.
↪000000             23.000000             23.000000             23.000000     ␣
↪         23.000000             23.000000             23.000000            23.
↪000000             23.000000             23.000000             23.000000
72074  1358034 2018-10-15       9.105263          522903.0          1.0         ␣
↪ 54.780488                     358279.0               0.0      0            54.
↪780488             54.780488             54.780488             54.780488     ␣
↪         54.780488             31.780488             54.780488            54.
↪780488             54.780488             54.780488             54.780488
72075  1358034 2018-10-22      26.934307         1004345.0          1.0         ␣
↪  9.105263                     522903.0               1.0      0             9.
↪105263              9.105263              9.105263              9.105263     ␣
↪          9.105263             -45.675225            -13.894737             9.
↪105263              9.105263              9.105263              9.105263
72076  1358034 2018-10-29     157.429487          433527.0          1.0         ␣
↪ 26.934307                    1004345.0               1.0      0             3.
↪934307             26.934307             26.934307             26.934307     ␣
↪         26.934307             17.829043            -27.846181            26.
↪934307             26.934307             26.934307             26.934307
72077  1358034 2018-11-05      50.813853          686717.0          1.0         ␣
↪157.429487                     433527.0               1.0      0           102.
↪648999            157.429487            157.429487            157.429487     ␣
↪        157.429487            130.495181            148.324224           134.
↪429487            157.429487            157.429487            157.429487
72078  1358034 2018-11-12      70.799270          650214.0          1.0         ␣
↪ 50.813853                     686717.0               1.0      0            41.
↪708590             27.813853             50.813853             50.813853     ␣
↪         50.813853           -106.615634             23.879546            -3.
↪966635             50.813853             50.813853             50.813853
72079  1358034 2018-11-19       0.000000               0.0          1.0         ␣
↪ 70.799270                     650214.0               1.0      1            43.
↪864964             16.018782             70.799270             70.799270     ␣
↪         70.799270             19.985417            -86.630217            61.
↪694007             47.799270             70.799270             70.799270
```

```
72080  1358034 2018-11-26      56.880851         1194396.0           1.0
↪  0.000000                        0.0                1.0       0             -157.
↪429487              -9.105263                0.000000                0.000000
↪            0.000000            -70.799270            -50.813853              -26.
↪934307             -54.780488            -23.000000               0.000000
```

[53]:

```
# Testing purposes:

df_first_weeks = df_all_weeks[(df_all_weeks['week'] == '2018-10-08') |
↪(df_all_weeks['week'] == '2018-10-15') | (df_all_weeks['week'] ==
↪'2018-10-22') | (df_all_weeks['week'] == '2018-10-29')]
```

[149]:

```
print(df_first_weeks['churn'].value_counts(normalize=True))
```

```
churn
0    0.998815
1    0.001185
Name: proportion, dtype: float64
```

6 Training the Churn Model (Rolling Prediction)

[244]:

```
def rolling_churn_prediction(df, features, label, model=None):
    """
    Perform week-to-week churn predictions using a rolling window approach.

    Parameters:
    - df: Aggregated user activity dataframe with churn labels.
    - features: List of feature column names.
    - label: Target column name (churn).
    - model: Machine learning model (default: Random Forest with
↪class_weight='balanced')

    Returns:
    - None (prints classification reports for each week).
    """
    if model is None:
        model = RandomForestClassifier(n_estimators=100, random_state=42,
↪class_weight='balanced')  # Handle class imbalance

    print(f"Model type: {type(model)}")

    weeks = sorted(df['week'].unique())  # Get sorted list of weeks
    f1_scores = []
    week_labels = []

    for i in range(len(weeks) - 1):  # Stop at second-to-last week (since we
↪predict next week)
```

```
        train_weeks = weeks[i:i + 1]  # Train up to current week
        test_week = weeks[i + 1]  # Predict for the next week

        train = df[df['week'].isin(train_weeks)]
        test = df[df['week'] == test_week]

        if test.empty:  # Skip if no data for the next week
            continue

        # Convert categorical features to numeric
        X_train = train[features]
        y_train = train[label]
        X_test = test[features]
        y_test = test[label]

        # X_train, X_test = X_train.align(X_test, join="left", axis=1,␣
↪fill_value=0)


        # Train the model
        model.fit(X_train, y_train)

        # Predict churn
        y_pred = model.predict(X_test)

        # Compute confusion matrix
        cm = confusion_matrix(y_test, y_pred)
        tn, fp, fn, tp = cm.ravel()

        # Compute specificity
        specificity = tn / (tn + fp) if (tn + fp) != 0 else 0

        # Generate classification report as a dictionary
        report_dict = classification_report(y_test, y_pred, output_dict=True,␣
↪zero_division=0)

        # Add specificity to the report
        report_dict['0']['specificity'] = specificity

        # Convert report to DataFrame for better display
        report_df = pd.DataFrame(report_dict).transpose()

        # Print evaluation results for each week
        print(f"Week {test_week}:")
        print(report_df)

        # Plot confusion matrix
```

```
        disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No␣
↪Churn', 'Churn'])
        disp.plot(cmap='Blues')
        plt.title(f"Confusion Matrix - Week {test_week}")
        plt.show()

        # F1 Score tracking
        f1 = f1_score(y_test, y_pred, zero_division=0)
        f1_scores.append(f1)
        week_labels.append(str(test_week))

        print("-" * 50)

    # Plot F1-scores over weeks
    if f1_scores:
        plt.figure(figsize=(10, 6))
        plt.plot(week_labels, f1_scores, marker='o')
        plt.title('F1-Score for Churn Prediction Over Weeks')
        plt.xlabel('Week')
        plt.ylabel('F1-Score')
        plt.xticks(rotation=45)
        plt.grid(True)
        plt.tight_layout()
        plt.show()
    else:
        print("No evaluations performed (possibly no data after filtering).")
```

[ ]:
```
model = lgb.LGBMClassifier(random_state=42)
features = ['prev_week_activity',
            'prev_week_session_duration',
            'prev_dummy_level',
            'rolling_diff_1_weeks',
            'rolling_diff_2_weeks',
            'rolling_diff_3_weeks',
            'rolling_diff_4_weeks',
            'rolling_diff_5_weeks',
            'rolling_diff_6_weeks',
            'rolling_diff_7_weeks',
            'rolling_diff_8_weeks',
            'prev_week_rolling_diff_7_days',
            'prev_week_rolling_diff_14_days',
            'prev_week_rolling_diff_21_days',
            'prev_week_rolling_diff_28_days'
            ]
label = 'churn'
```

```
rolling_churn_prediction(df = df_all_weeks, features=features, label=label,␣
    ↪model=model)
```

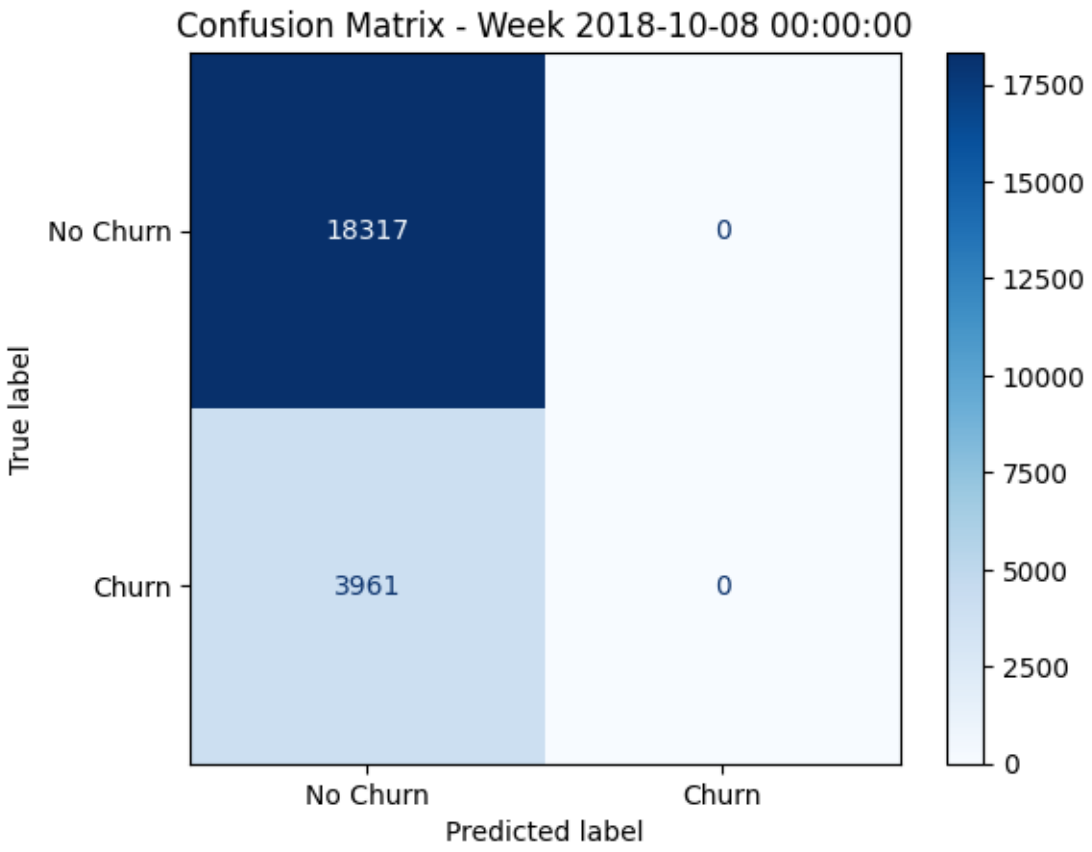
```
Model type: <class 'lightgbm.sklearn.LGBMClassifier'>
[LightGBM] [Warning] Contains only one class
[LightGBM] [Info] Number of positive: 0, number of negative: 22278
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000189 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 2038
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 9
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.000000 -> initscore=-34.538776
[LightGBM] [Info] Start training from score -34.538776
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
```

```
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
```

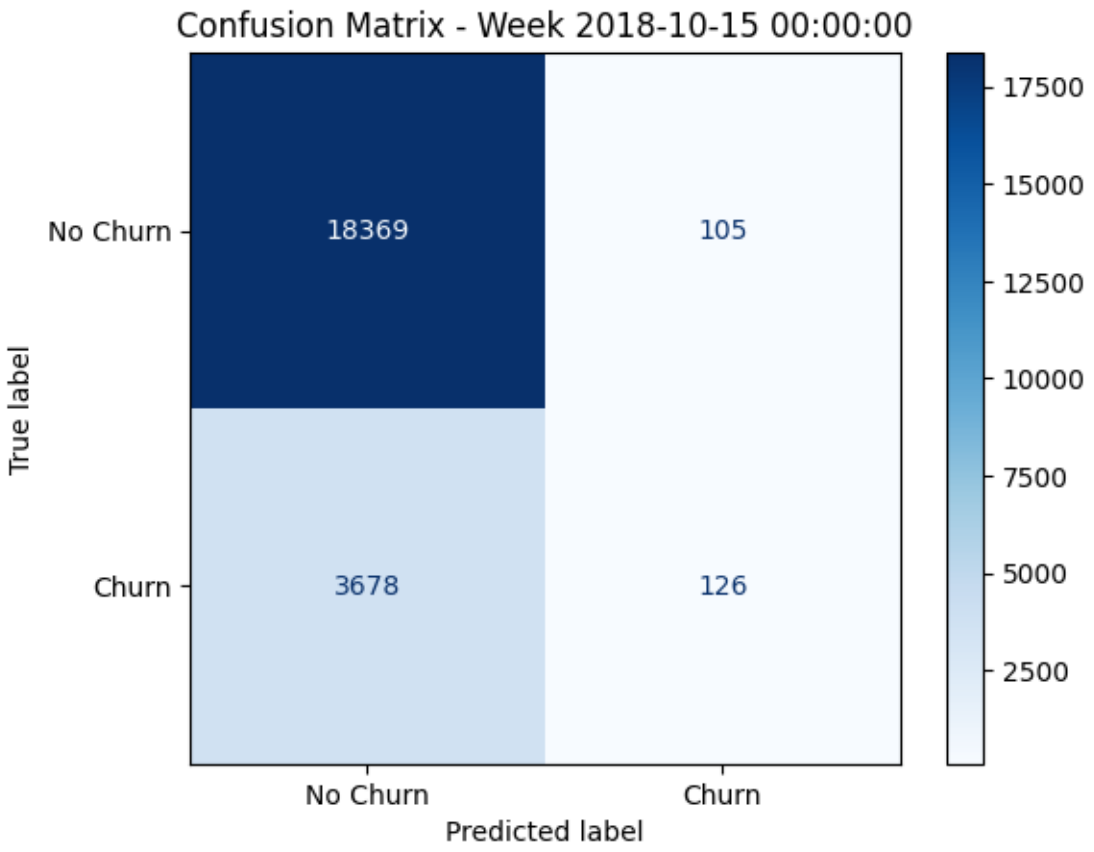
```
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
```

```
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
```

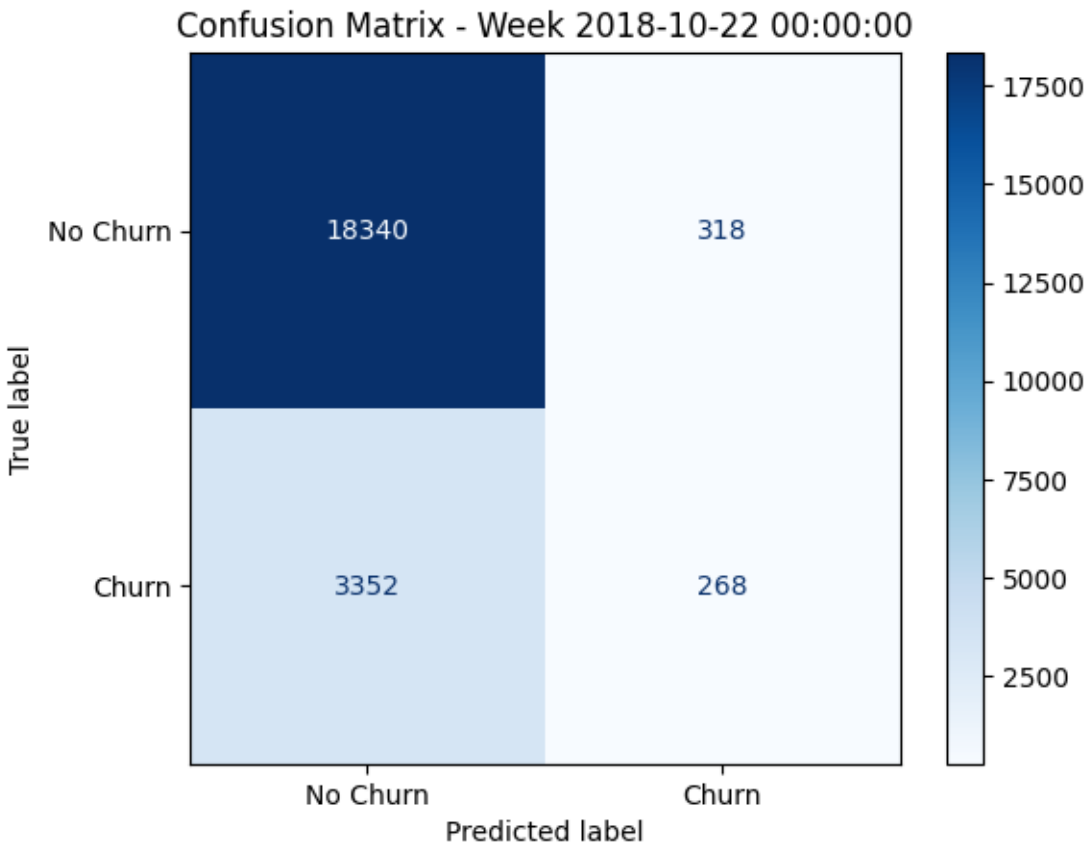
```
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
[LightGBM] [Warning] Stopped training because there are no more leaves that meet
the split requirements
Week 2018-10-08 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.822201  1.000000  0.902426  18317.000000     1.000000
1              0.000000  0.000000  0.000000   3961.000000          NaN
accuracy       0.822201  0.822201  0.822201      0.822201     0.822201
macro avg      0.411101  0.500000  0.451213  22278.000000          NaN
weighted avg   0.676015  0.822201  0.741976  22278.000000          NaN
```



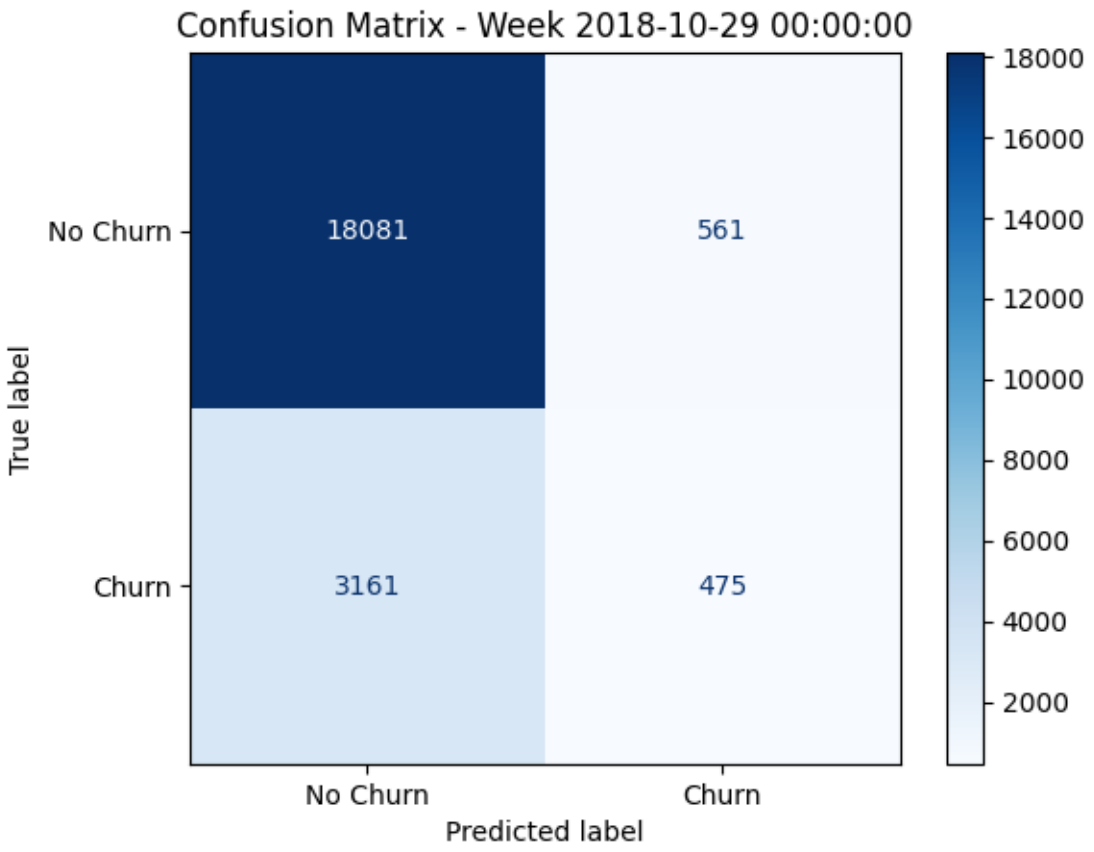

```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3961, number of negative: 18317
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.001477 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.177799 -> initscore=-1.531333
[LightGBM] [Info] Start training from score -1.531333
Week 2018-10-15 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.833175  0.994316  0.906641  18474.000000     0.994316
1              0.545455  0.033123  0.062454   3804.000000          NaN
accuracy       0.830191  0.830191  0.830191      0.830191     0.830191
macro avg      0.689315  0.513720  0.484547  22278.000000          NaN
weighted avg   0.784046  0.830191  0.762495  22278.000000          NaN
```



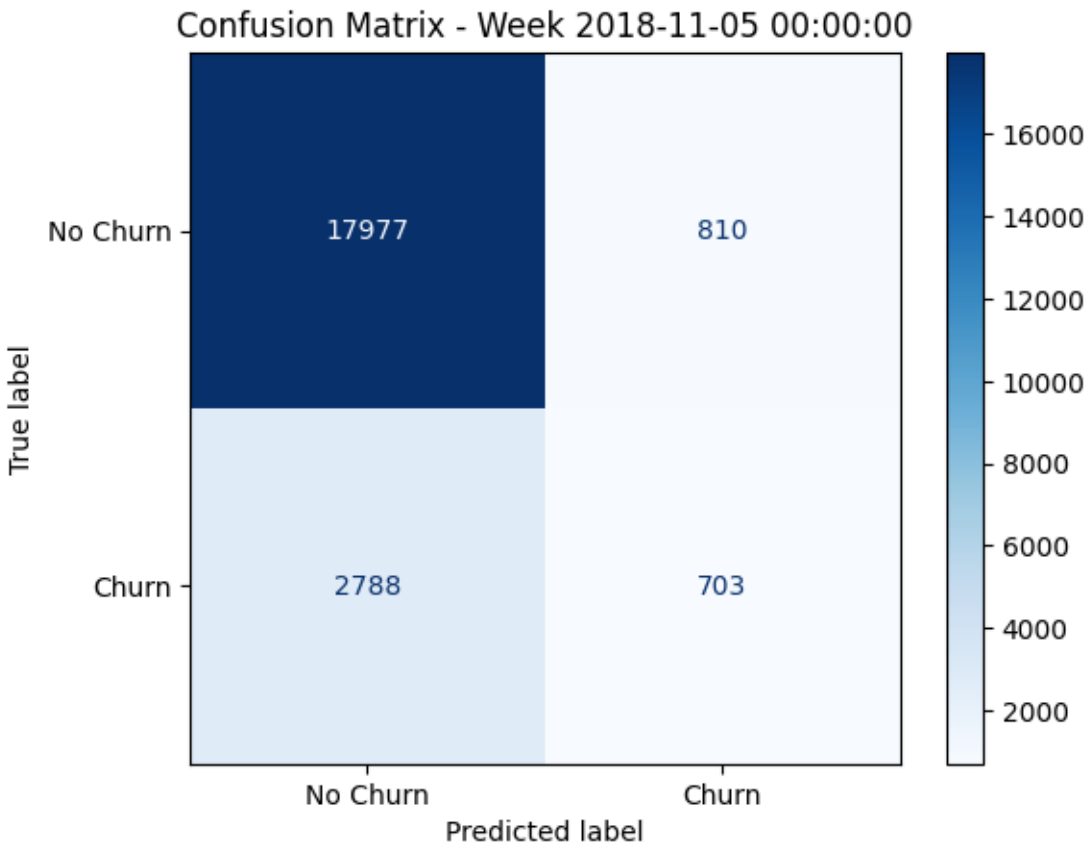

```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3804, number of negative: 18474
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.001069 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.170751 -> initscore=-1.580311
[LightGBM] [Info] Start training from score -1.580311
Week 2018-10-22 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.845473  0.982956  0.909046  18658.000000     0.982956
1              0.457338  0.074033  0.127437   3620.000000          NaN
accuracy       0.835263  0.835263  0.835263      0.835263     0.835263
macro avg      0.651405  0.528495  0.518241  22278.000000          NaN
weighted avg   0.782404  0.835263  0.782041  22278.000000          NaN
```



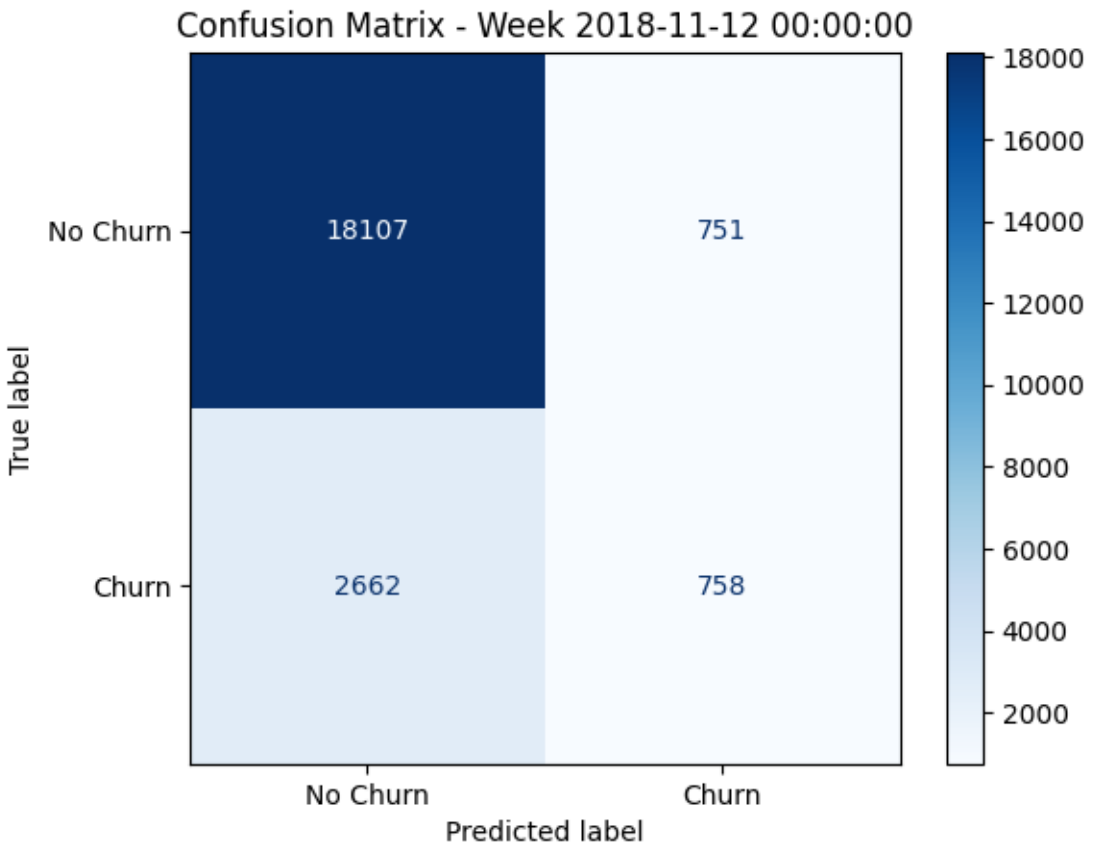

```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3620, number of negative: 18658
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000529 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.162492 -> initscore=-1.639801
[LightGBM] [Info] Start training from score -1.639801
Week 2018-10-29 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.851191  0.969907  0.906679  18642.000000     0.969907
1              0.458494  0.130638  0.203339   3636.000000          NaN
accuracy       0.832929  0.832929  0.832929      0.832929     0.832929
macro avg      0.654843  0.550272  0.555009  22278.000000          NaN
weighted avg   0.787099  0.832929  0.791887  22278.000000          NaN
```



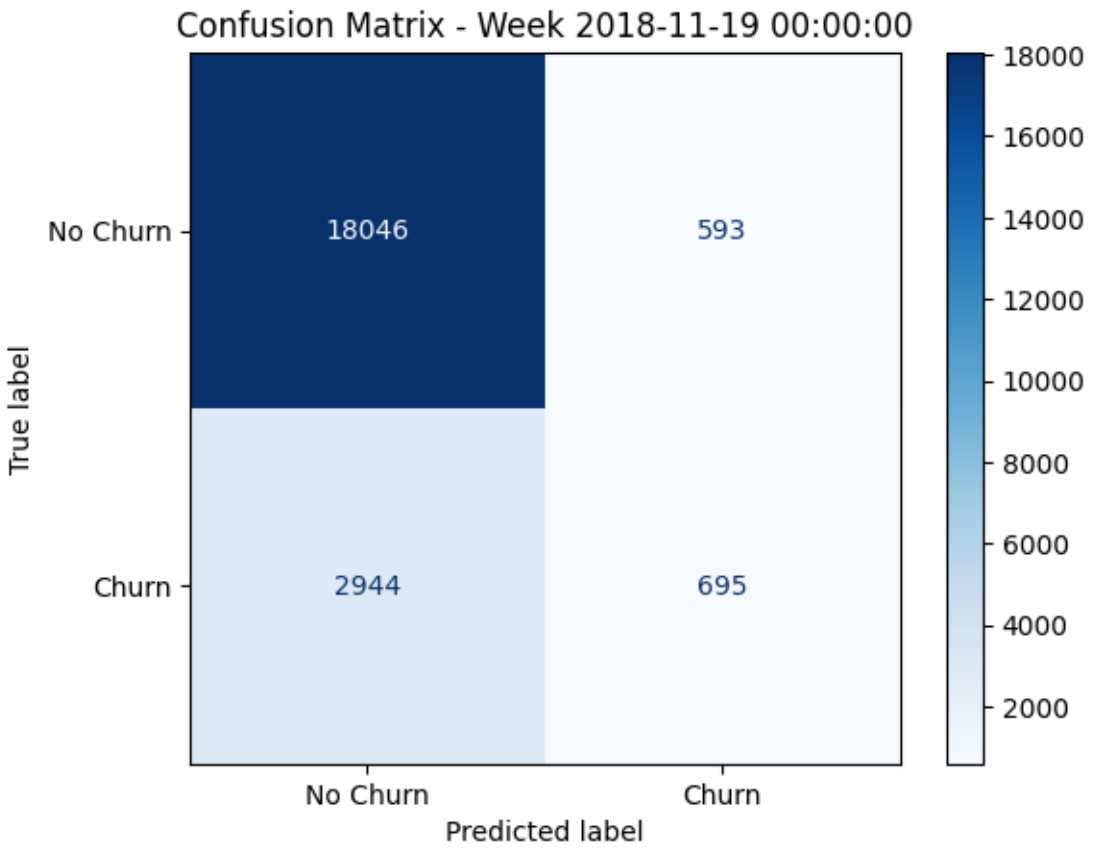

```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3636, number of negative: 18642
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000453 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.163210 -> initscore=-1.634533
[LightGBM] [Info] Start training from score -1.634533
Week 2018-11-05 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.865736  0.956885  0.909031  18787.000000     0.956885
1              0.464640  0.201375  0.280975   3491.000000          NaN
accuracy       0.838495  0.838495  0.838495      0.838495     0.838495
macro avg      0.665188  0.579130  0.595003  22278.000000          NaN
weighted avg   0.802883  0.838495  0.810614  22278.000000          NaN
```



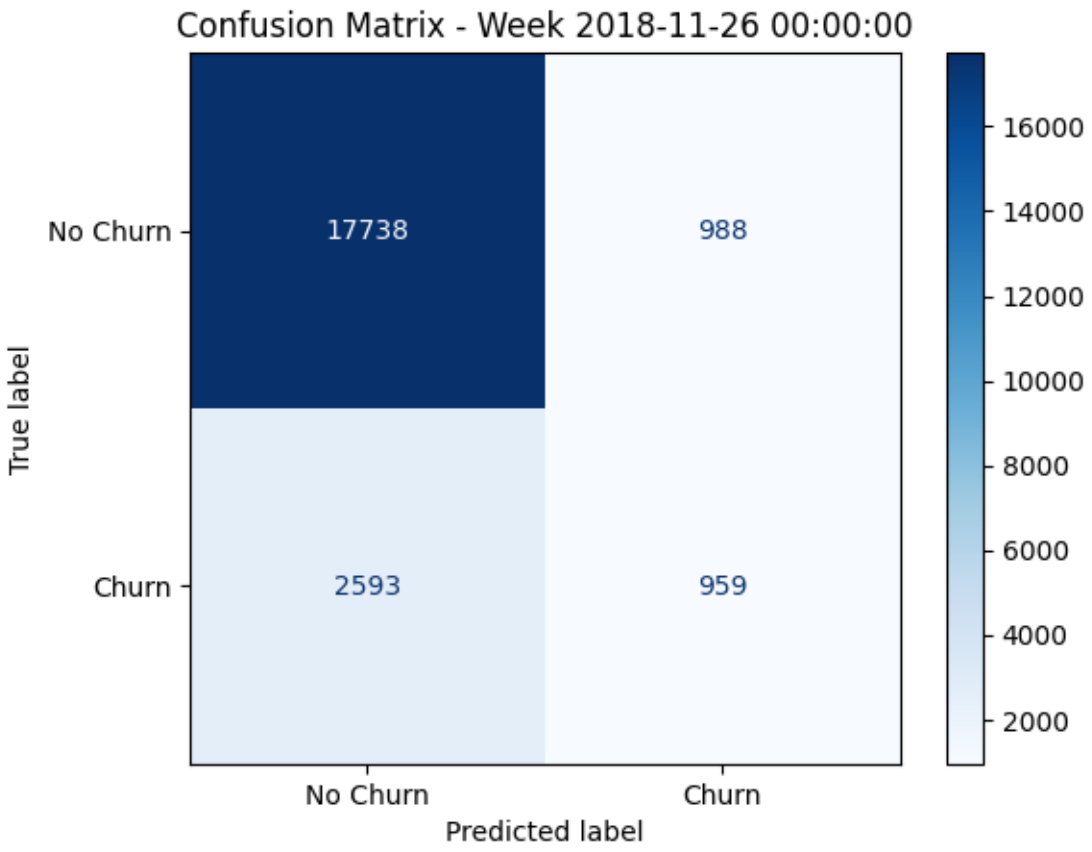

```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3491, number of negative: 18787
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000370 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.156702 -> initscore=-1.682977
[LightGBM] [Info] Start training from score -1.682977
Week 2018-11-12 00:00:00:
              precision    recall  f1-score     support  specificity
0              0.871828  0.960176  0.913872  18858.0000     0.960176
1              0.502319  0.221637  0.307567   3420.0000          NaN
accuracy       0.846800  0.846800  0.846800      0.8468     0.846800
macro avg      0.687074  0.590907  0.610720  22278.0000          NaN
weighted avg   0.815103  0.846800  0.820795  22278.0000          NaN
```

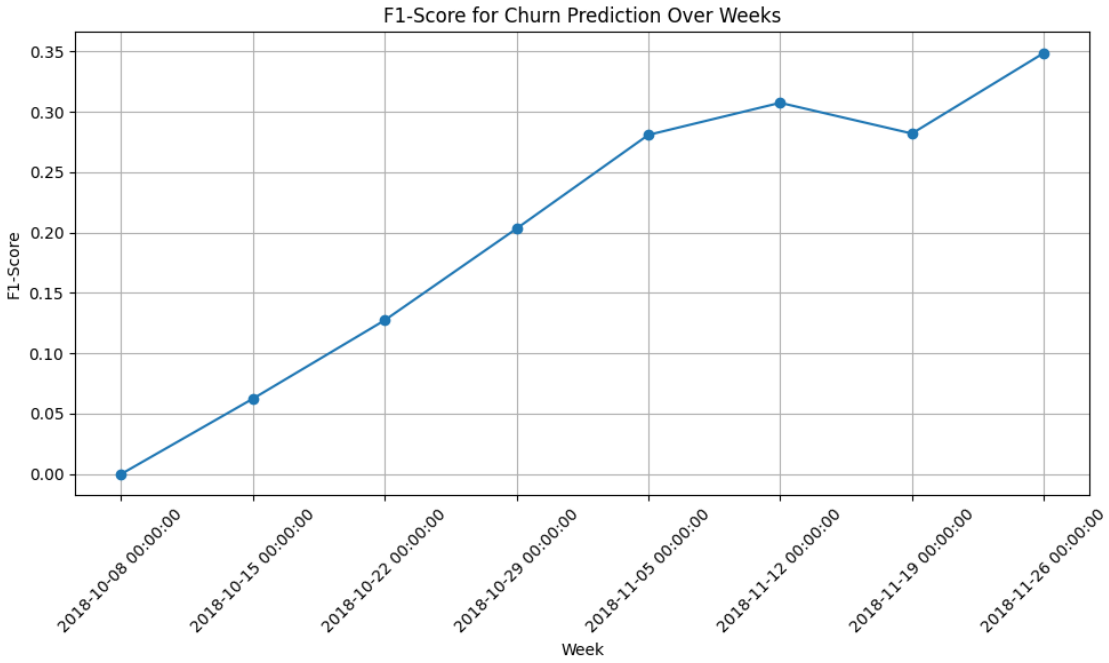



```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3420, number of negative: 18858
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000288 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.153515 -> initscore=-1.707297
[LightGBM] [Info] Start training from score -1.707297
Week 2018-11-19 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.859743  0.968185  0.910747  18639.000000     0.968185
1              0.539596  0.190987  0.282119   3639.000000          NaN
accuracy       0.841234  0.841234  0.841234      0.841234     0.841234
macro avg      0.699670  0.579586  0.596433  22278.000000          NaN
weighted avg   0.807448  0.841234  0.808064  22278.000000          NaN
```




```
--------------------------------------------------
[LightGBM] [Info] Number of positive: 3639, number of negative: 18639
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.000406 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 3572
[LightGBM] [Info] Number of data points in the train set: 22278, number of used
features: 15
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.163345 -> initscore=-1.633547
[LightGBM] [Info] Start training from score -1.633547
Week 2018-11-26 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.872461  0.947239  0.908313  18726.000000     0.947239
1              0.492553  0.269989  0.348791   3552.000000          NaN
accuracy       0.839258  0.839258  0.839258      0.839258     0.839258
macro avg      0.682507  0.608614  0.628552  22278.000000          NaN
weighted avg   0.811888  0.839258  0.819103  22278.000000          NaN
```


Confusion Matrix - Week 2018-11-26 00:00:00
No Churn
Churn
True label
17738
988
2593
959
No Churn
Churn
Predicted label
16000
14000
12000
10000
8000
6000
4000
2000

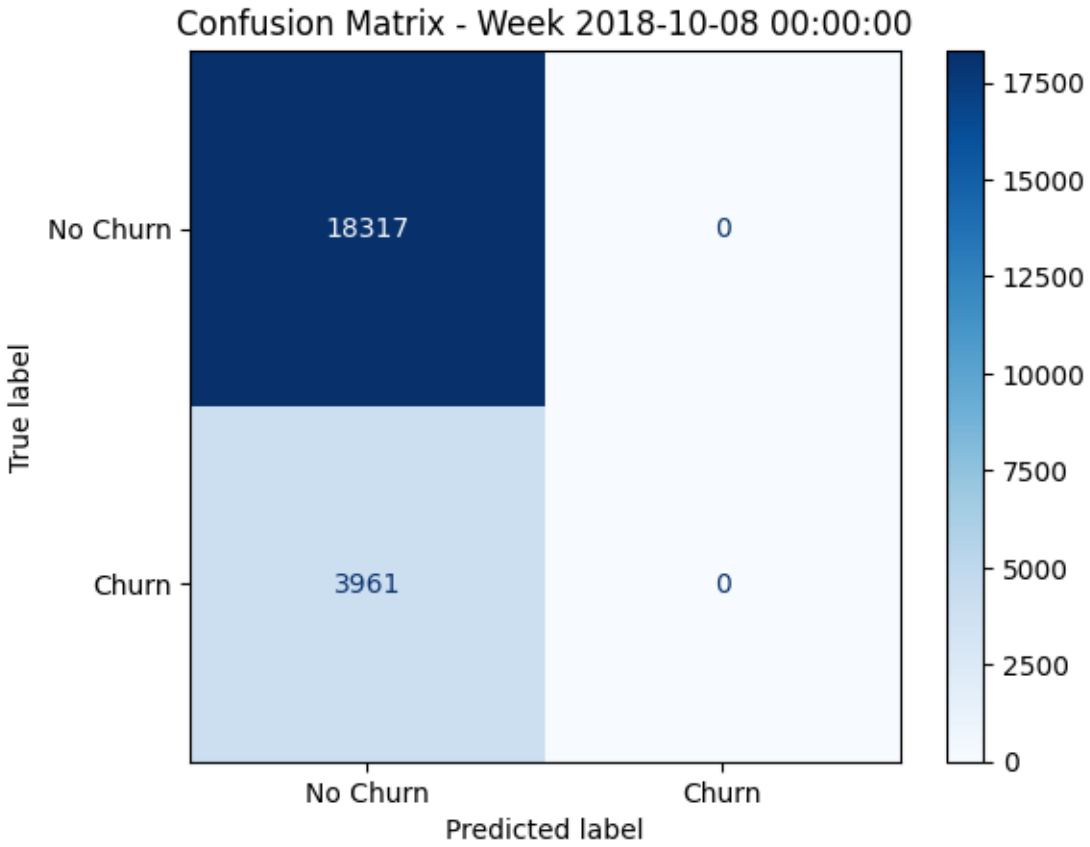

F1-Score for Churn Prediction Over Weeks
F1-Score
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
2018-10-08 00:00:00
2018-10-15 00:00:00
2018-10-22 00:00:00
2018-10-29 00:00:00
2018-11-05 00:00:00
2018-11-12 00:00:00
2018-11-19 00:00:00
2018-11-26 00:00:00
Week

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[245], line 25
     22 # Apply shift for previous week rolling diffs
     23 for window in window_sizes:
     24     df_all_weeks[f'prev_week_rolling_diff_{window}_days'] = (
---> 25         ␣
  ↪df_all_weeks.groupby('userId', group_keys=False)[f'rolling_diff_{window}_days ]
     26          .shift(1)  # Shift values down by one week per user
     27          .fillna(0)
     28     )

File ~/Documents/Seminar/.venv/lib/python3.13/site-packages/pandas/core/groupby
  ↪generic.py:1951, in DataFrameGroupBy.__getitem__(self, key)
   1944 if isinstance(key, tuple) and len(key) > 1:
   1945     # if len == 1, then it becomes a SeriesGroupBy and this is actually
   1946     # valid syntax, so don't raise
   1947     raise ValueError(
   1948         "Cannot subset columns with a tuple with more than one element. "
   1949         "Use a list instead."
   1950     )
-> 1951 return super().__getitem__(key)

File ~/Documents/Seminar/.venv/lib/python3.13/site-packages/pandas/core/base.py
  ↪244, in SelectionMixin.__getitem__(self, key)
    242 else:
    243     if key not in self.obj:
--> 244         raise KeyError(f"Column not found: {key}")
    245     ndim = self.obj[key].ndim
    246     return self._gotitem(key, ndim=ndim)

KeyError: 'Column not found: rolling_diff_1_days'
```

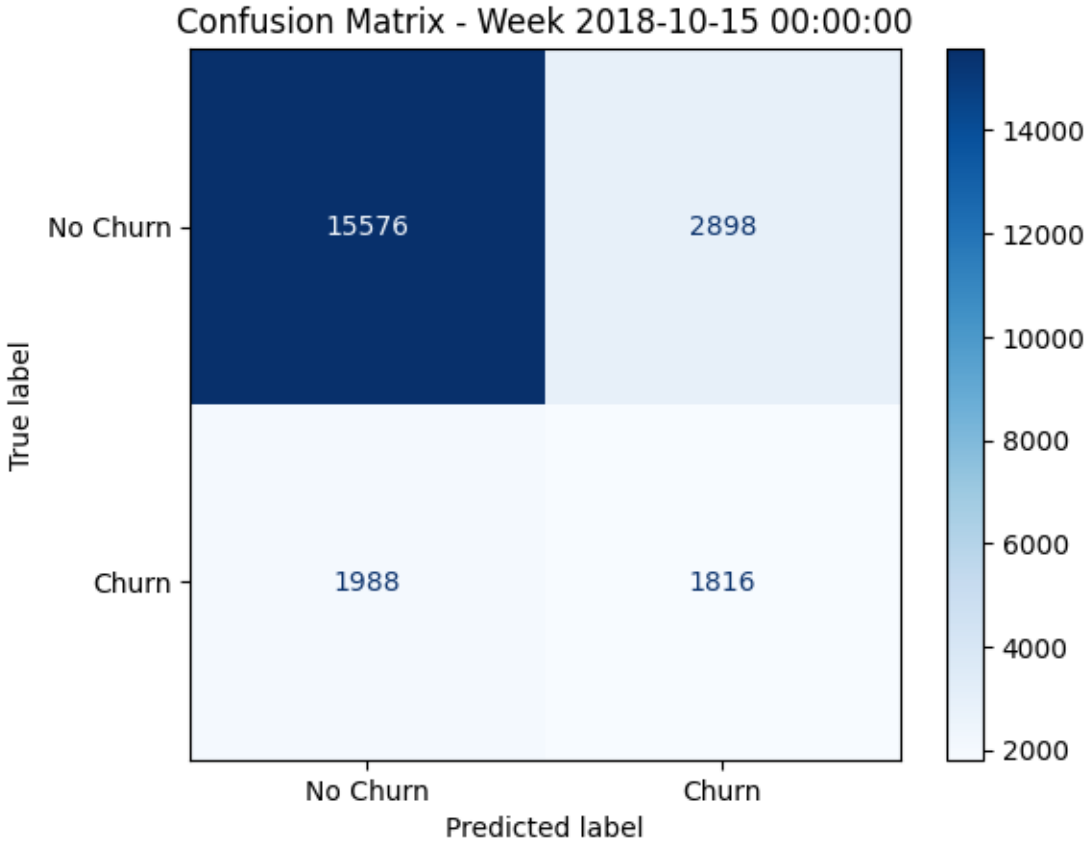
[246]:
```
# Fitting a XGB model
model = XGBClassifier(n_estimators=100, random_state=42, base_score=0.5,␣
  ↪scale_pos_weight=10)
features = ['prev_week_activity',
            'prev_week_session_duration',
            'prev_dummy_level',
            'rolling_diff_1_weeks',
            'rolling_diff_2_weeks',
            'rolling_diff_3_weeks',
            'rolling_diff_4_weeks',
            'rolling_diff_5_weeks',
```

```
            'rolling_diff_6_weeks',
            'rolling_diff_7_weeks',
            'rolling_diff_8_weeks',
            'prev_week_rolling_diff_7_days',
            'prev_week_rolling_diff_14_days',
            'prev_week_rolling_diff_21_days',
            'prev_week_rolling_diff_28_days'
           ]
label = 'churn'

rolling_churn_prediction(df = df_all_weeks, features=features, label=label,␣
 ↪model=model)
```

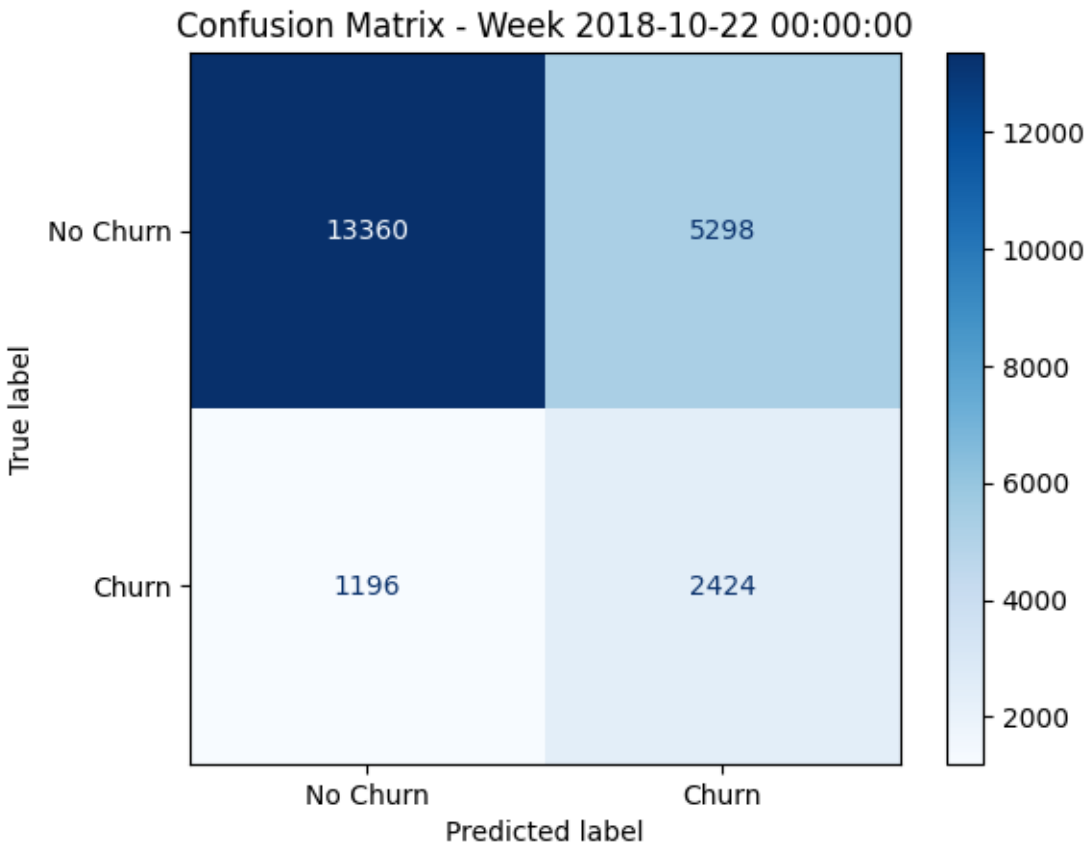
```
Model type: <class 'xgboost.sklearn.XGBClassifier'>
Week 2018-10-08 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.822201  1.000000  0.902426  18317.000000     1.000000
1              0.000000  0.000000  0.000000   3961.000000          NaN
accuracy       0.822201  0.822201  0.822201      0.822201     0.822201
macro avg      0.411101  0.500000  0.451213  22278.000000          NaN
weighted avg   0.676015  0.822201  0.741976  22278.000000          NaN
```



```
------------------------------------------------
Week 2018-10-15 00:00:00:
              precision    recall  f1-score      support  specificity
0              0.886814  0.843131  0.864421  18474.00000     0.843131
1              0.385235  0.477392  0.426391   3804.00000          NaN
accuracy       0.780680  0.780680  0.780680      0.78068     0.780680
macro avg      0.636025  0.660262  0.645406  22278.00000          NaN
weighted avg   0.801169  0.780680  0.789627  22278.00000          NaN
```

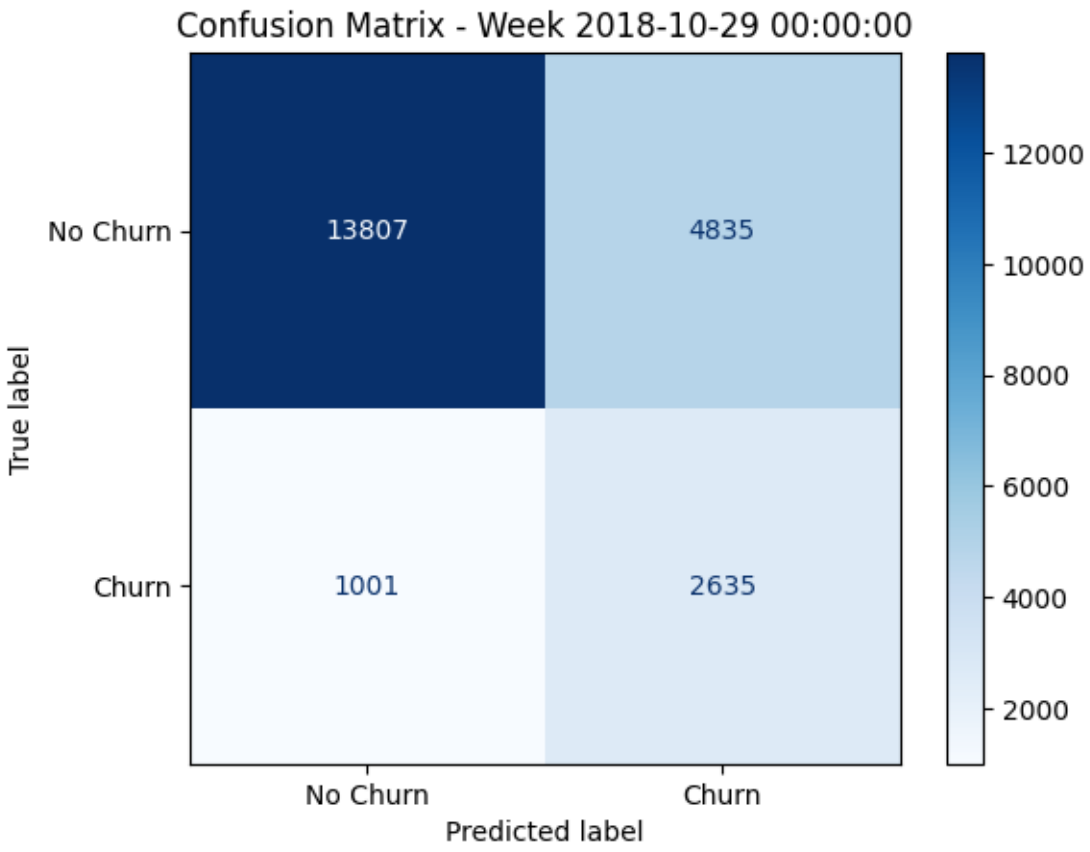



```
------------------------------------------------
Week 2018-10-22 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.917835  0.716047  0.804480  18658.000000     0.716047
1              0.313908  0.669613  0.427438   3620.000000          NaN
accuracy       0.708502  0.708502  0.708502      0.708502     0.708502
macro avg      0.615871  0.692830  0.615959  22278.000000          NaN
weighted avg   0.819701  0.708502  0.743214  22278.000000          NaN
```



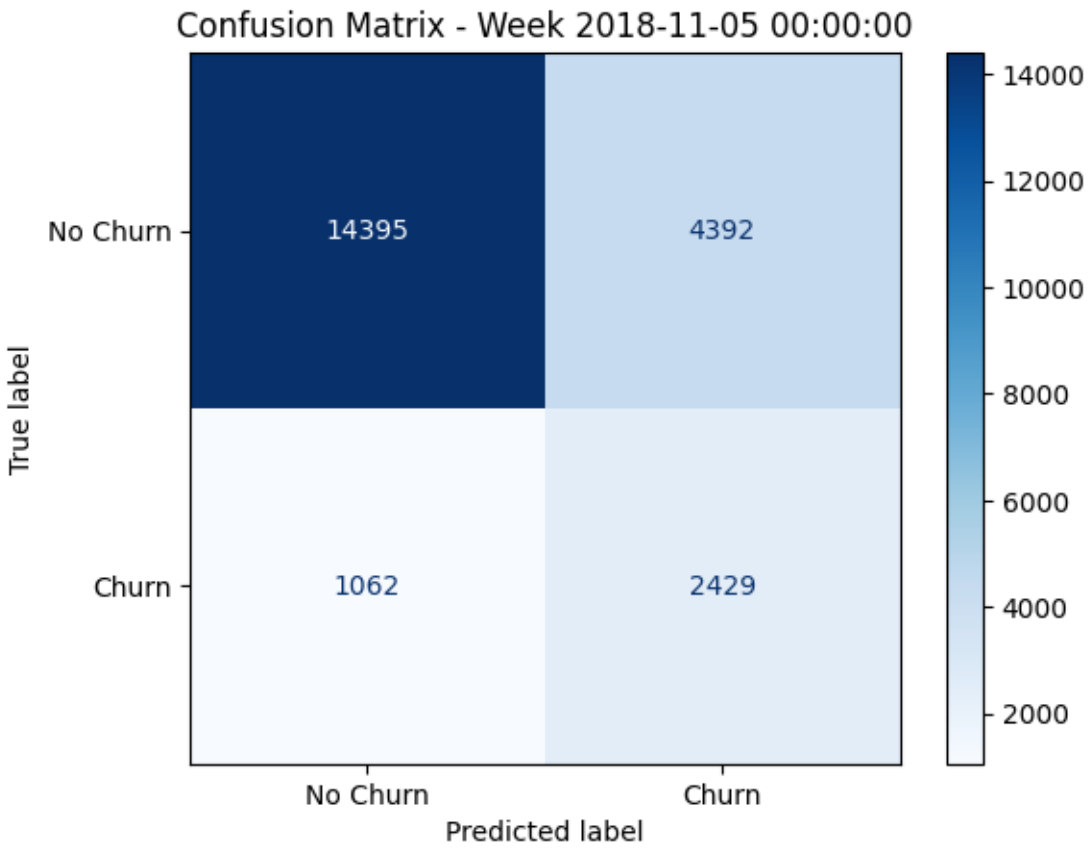
Week 2018-10-29 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.932401	0.740639	0.825531	18642.000000	0.740639
1	0.352744	0.724697	0.474518	3636.000000	NaN
accuracy	0.738038	0.738038	0.738038	0.738038	0.738038
macro avg	0.642573	0.732668	0.650024	22278.000000	NaN
weighted avg	0.837795	0.738038	0.768242	22278.000000	NaN



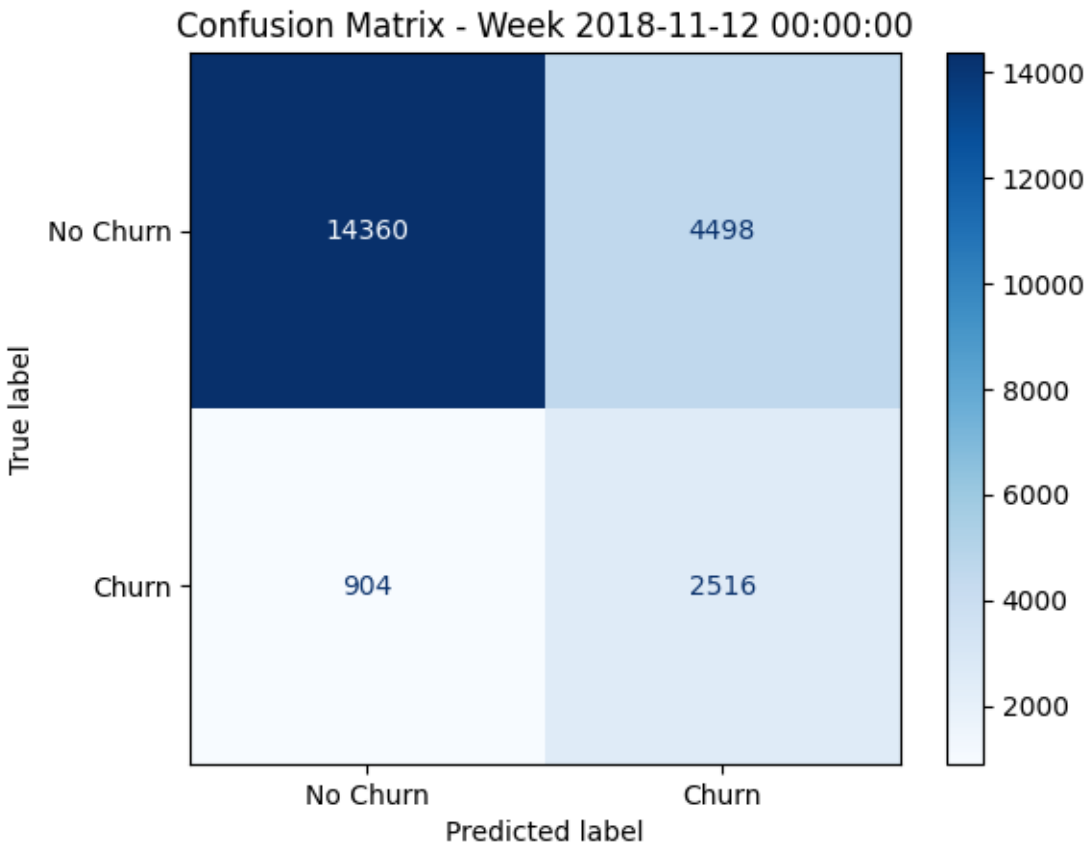
Week 2018-11-05 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.931293	0.766221	0.840731	18787.000000	0.766221
1	0.356106	0.695789	0.471102	3491.000000	NaN
accuracy	0.755184	0.755184	0.755184	0.755184	0.755184
macro avg	0.643700	0.731005	0.655916	22278.000000	NaN
weighted avg	0.841160	0.755184	0.782810	22278.000000	NaN



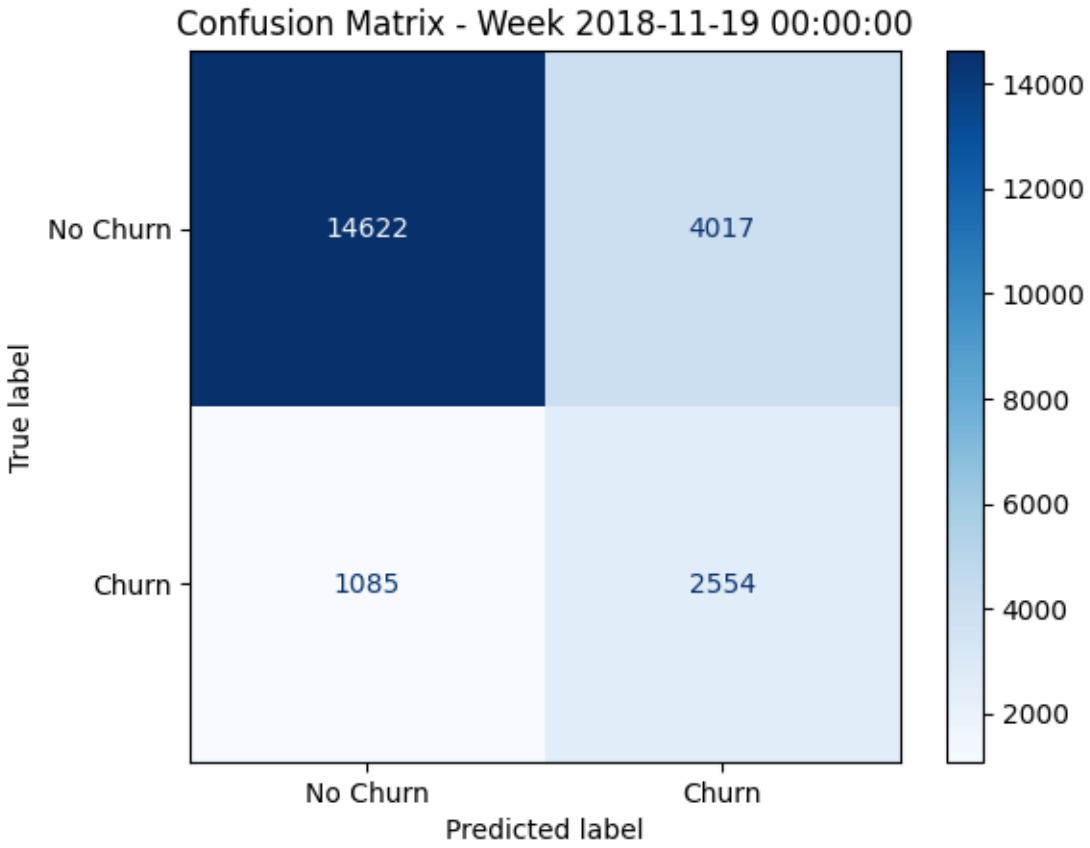
Week 2018-11-12 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.940776	0.761481	0.841686	18858.000000	0.761481
1	0.358711	0.735673	0.482270	3420.000000	NaN
accuracy	0.757519	0.757519	0.757519	0.757519	0.757519
macro avg	0.649743	0.748577	0.661978	22278.000000	NaN
weighted avg	0.851420	0.757519	0.786510	22278.000000	NaN



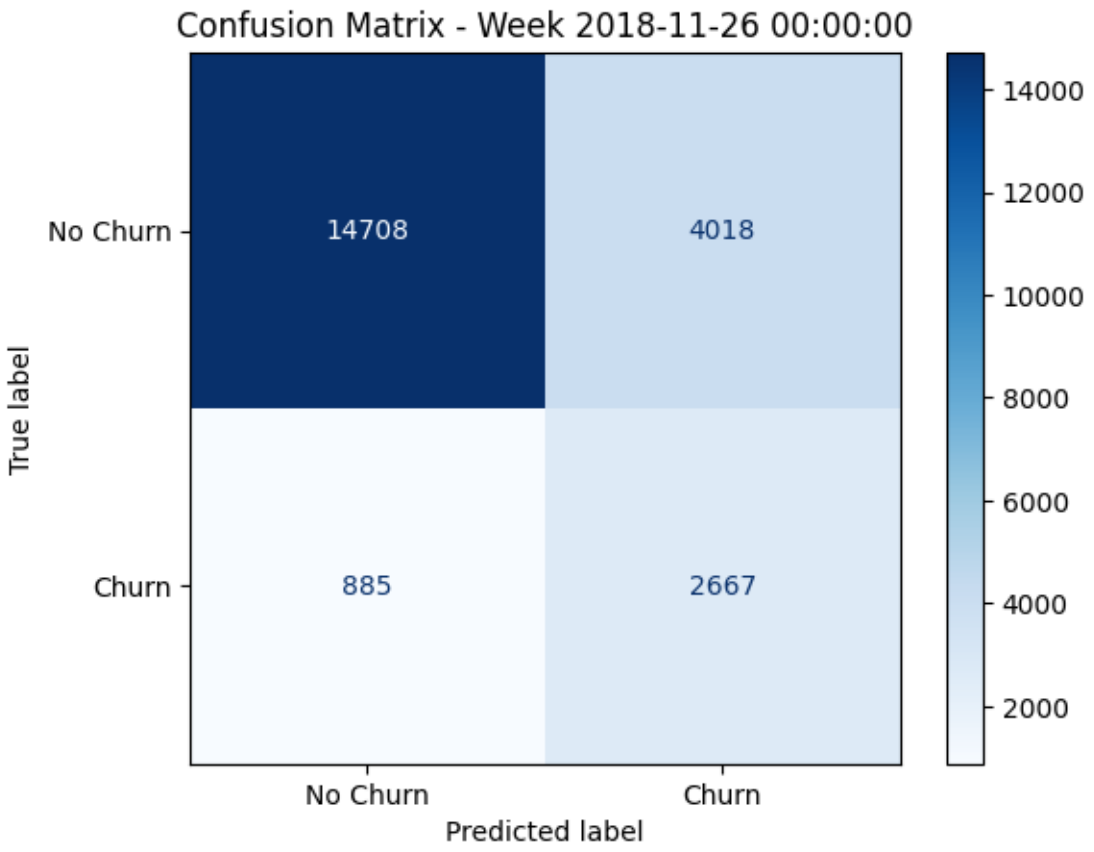
Week 2018-11-19 00:00:00:

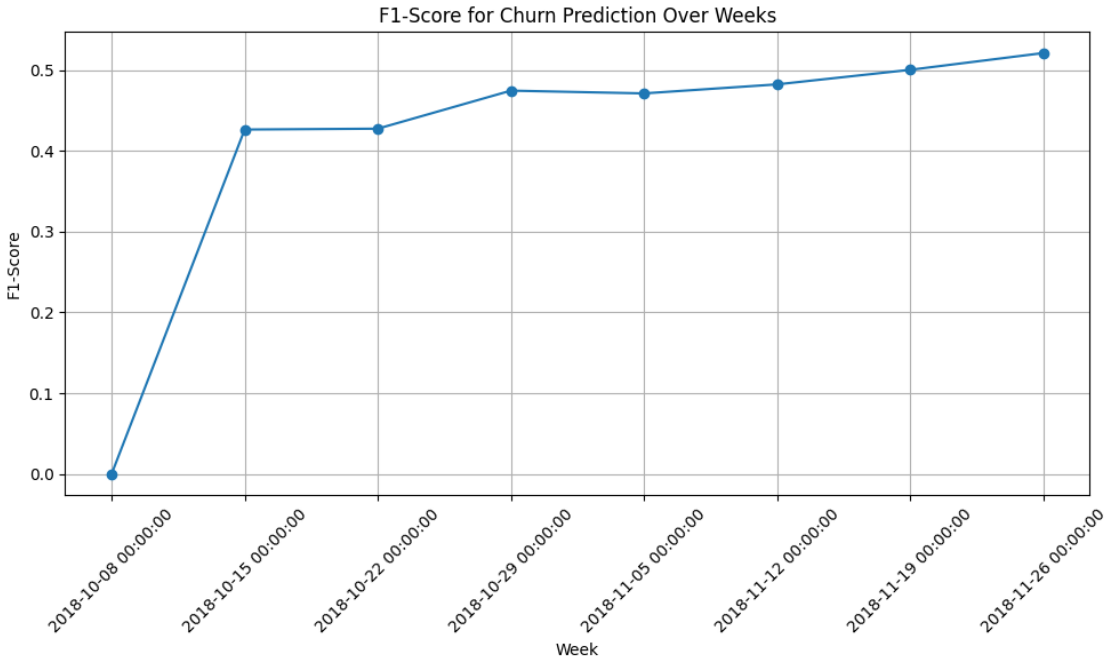
	precision	recall	f1-score	support	specificity
0	0.930923	0.784484	0.851453	18639.000000	0.784484
1	0.388678	0.701841	0.500294	3639.000000	NaN
accuracy	0.770985	0.770985	0.770985	0.770985	0.770985
macro avg	0.659800	0.743163	0.675873	22278.000000	NaN
weighted avg	0.842350	0.770985	0.794093	22278.000000	NaN



Week 2018-11-26 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.943244	0.785432	0.857135	18726.000000	0.785432
1	0.398953	0.750845	0.521051	3552.000000	NaN
accuracy	0.779917	0.779917	0.779917	0.779917	0.779917
macro avg	0.671098	0.768138	0.689093	22278.000000	NaN
weighted avg	0.856462	0.779917	0.803549	22278.000000	NaN


Confusion Matrix - Week 2018-11-26 00:00:00
No Churn
Churn
True label
14708
4018
885
2667
No Churn
Churn
Predicted label
14000
12000
10000
8000
6000
4000
2000


F1-Score for Churn Prediction Over Weeks
F1-Score
0.0
0.1
0.2
0.3
0.4
0.5
2018-10-08 00:00:00
2018-10-15 00:00:00
2018-10-22 00:00:00
2018-10-29 00:00:00
2018-11-05 00:00:00
2018-11-12 00:00:00
2018-11-19 00:00:00
2018-11-26 00:00:00
Week

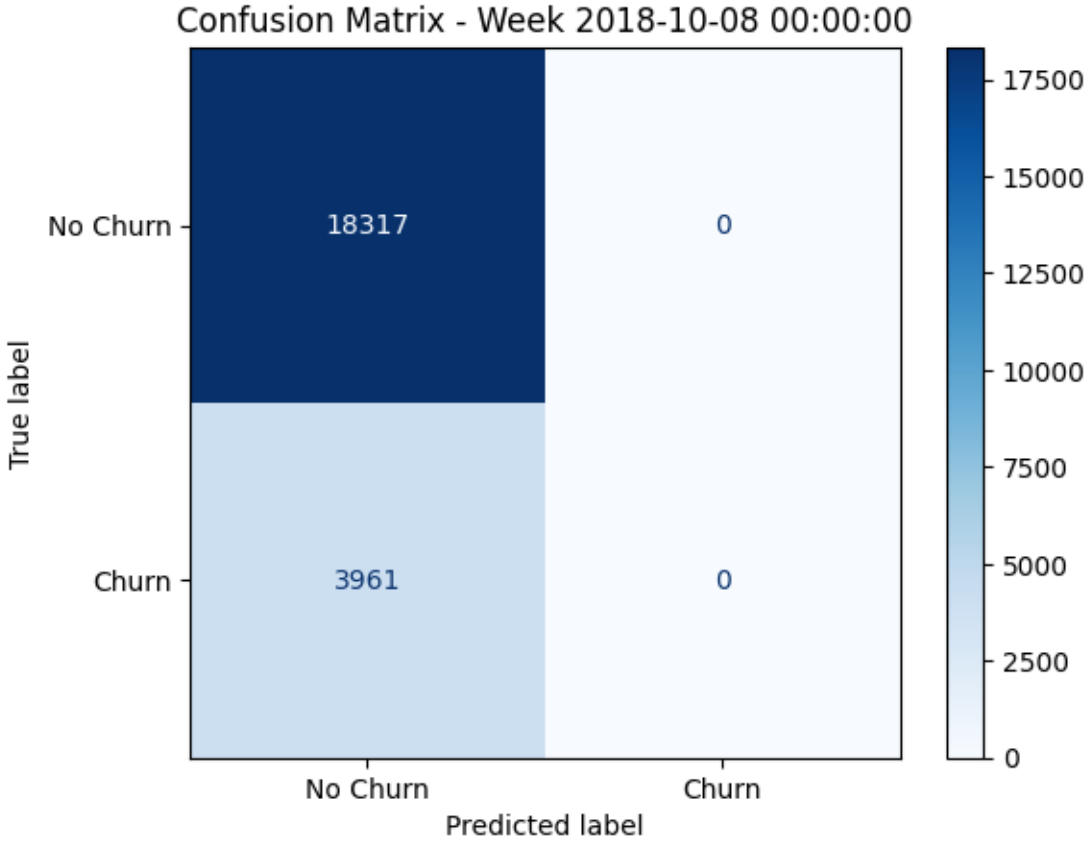
[247]:
```
print(df_all_weeks[features + [label]].corr()[label].
 ↪sort_values(ascending=False))
```

```
churn                              1.000000
prev_week_session_duration         0.273998
rolling_diff_4_weeks               0.156740
rolling_diff_5_weeks               0.155946
rolling_diff_3_weeks               0.155398
rolling_diff_2_weeks               0.153361
rolling_diff_6_weeks               0.152086
rolling_diff_7_weeks               0.149336
rolling_diff_8_weeks               0.147695
prev_week_activity                 0.142876
rolling_diff_1_weeks               0.138172
prev_dummy_level                  -0.037114
prev_week_rolling_diff_7_days     -0.040808
prev_week_rolling_diff_14_days    -0.069184
prev_week_rolling_diff_21_days    -0.079749
prev_week_rolling_diff_28_days    -0.082911
Name: churn, dtype: float64
```

[248]:
```
# Fitting a Random forest model
features = ['prev_week_activity',
            'prev_week_session_duration',
            'prev_dummy_level',
            'rolling_diff_1_weeks',
            'rolling_diff_2_weeks',
            'rolling_diff_3_weeks',
            'rolling_diff_4_weeks',
            'rolling_diff_5_weeks',
            'rolling_diff_6_weeks',
            'rolling_diff_7_weeks',
            'rolling_diff_8_weeks',
            'prev_week_rolling_diff_7_days',
            'prev_week_rolling_diff_14_days',
            'prev_week_rolling_diff_21_days',
            'prev_week_rolling_diff_28_days'
           ]
label = 'churn'
rolling_churn_prediction(df = df_all_weeks, features=features, label=label,␣
 ↪model=None)
```

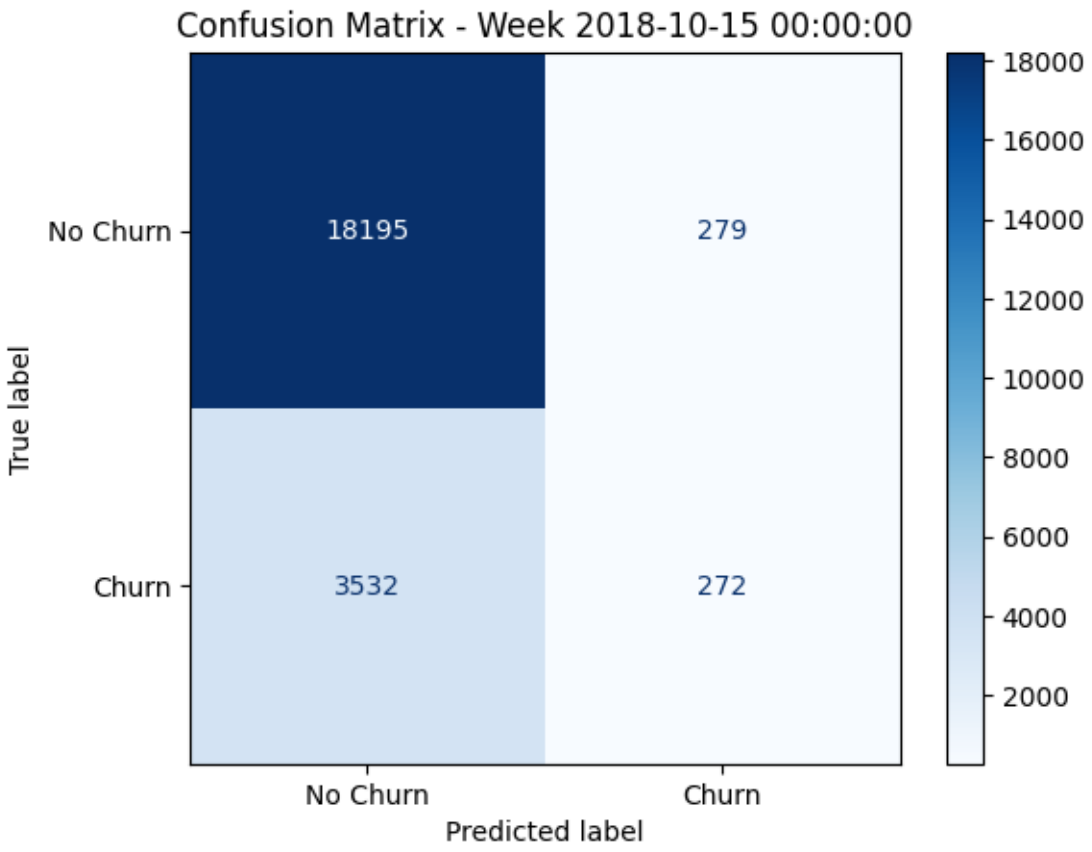
```
Model type: <class 'sklearn.ensemble._forest.RandomForestClassifier'>
Week 2018-10-08 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.822201  1.000000  0.902426  18317.000000     1.000000
```

1	0.000000	0.000000	0.000000	3961.000000	NaN
accuracy	0.822201	0.822201	0.822201	0.822201	0.822201
macro avg	0.411101	0.500000	0.451213	22278.000000	NaN
weighted avg	0.676015	0.822201	0.741976	22278.000000	NaN



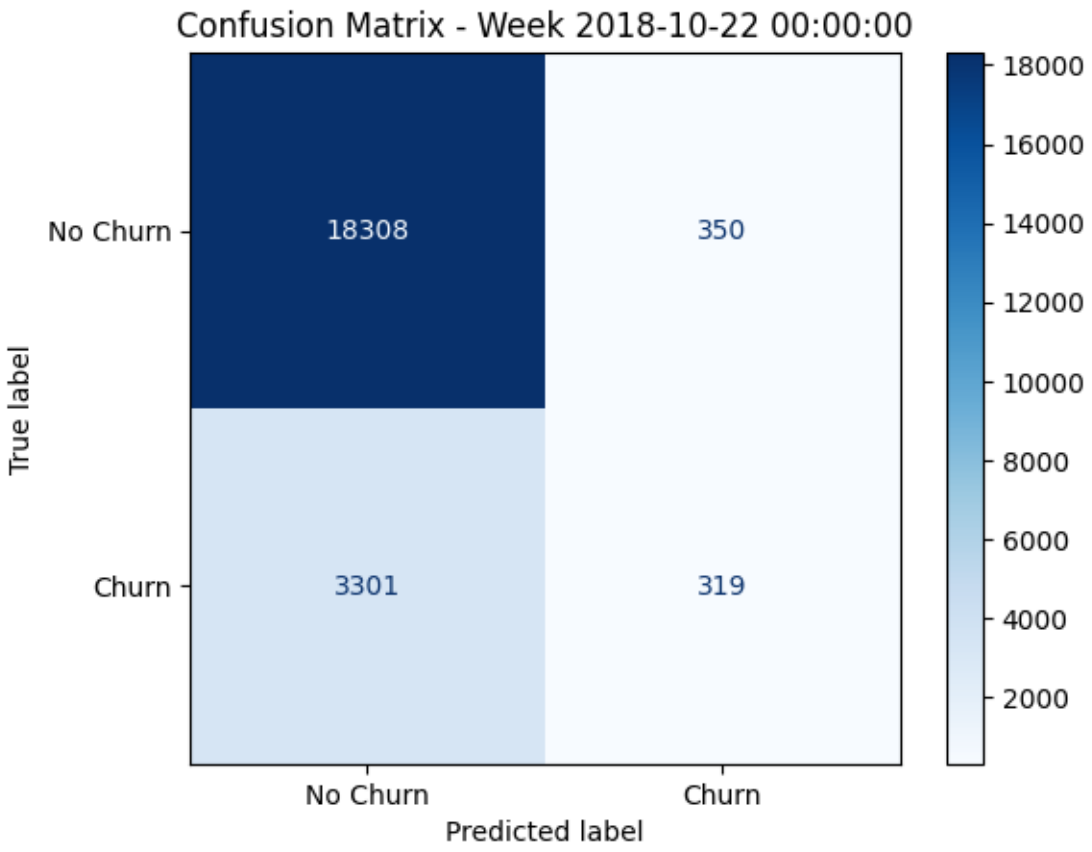
Week 2018-10-15 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.837437	0.984898	0.905201	18474.000000	0.984898
1	0.493648	0.071504	0.124914	3804.000000	NaN
accuracy	0.828934	0.828934	0.828934	0.828934	0.828934
macro avg	0.665543	0.528201	0.515058	22278.000000	NaN
weighted avg	0.778735	0.828934	0.771966	22278.000000	NaN



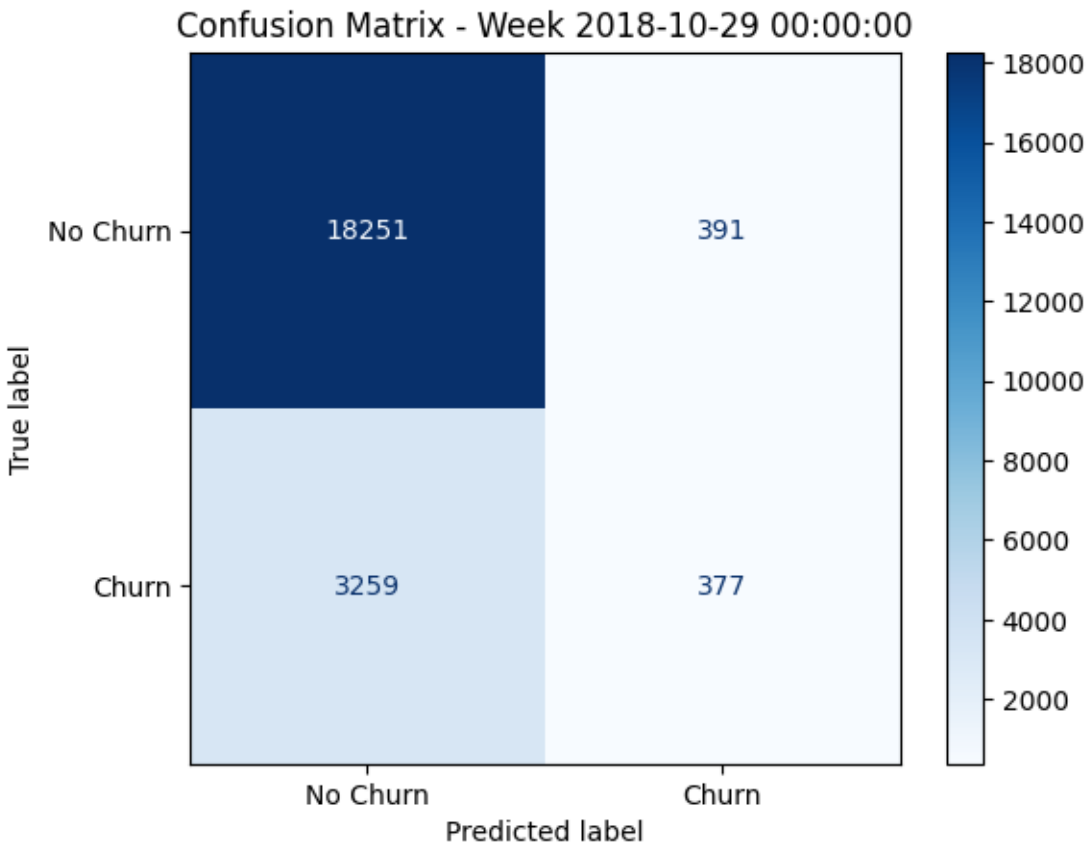
Week 2018-10-22 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.847240	0.981241	0.909330	18658.000000	0.981241
1	0.476831	0.088122	0.148753	3620.000000	NaN
accuracy	0.836116	0.836116	0.836116	0.836116	0.836116
macro avg	0.662035	0.534681	0.529041	22278.000000	NaN
weighted avg	0.787051	0.836116	0.785742	22278.000000	NaN



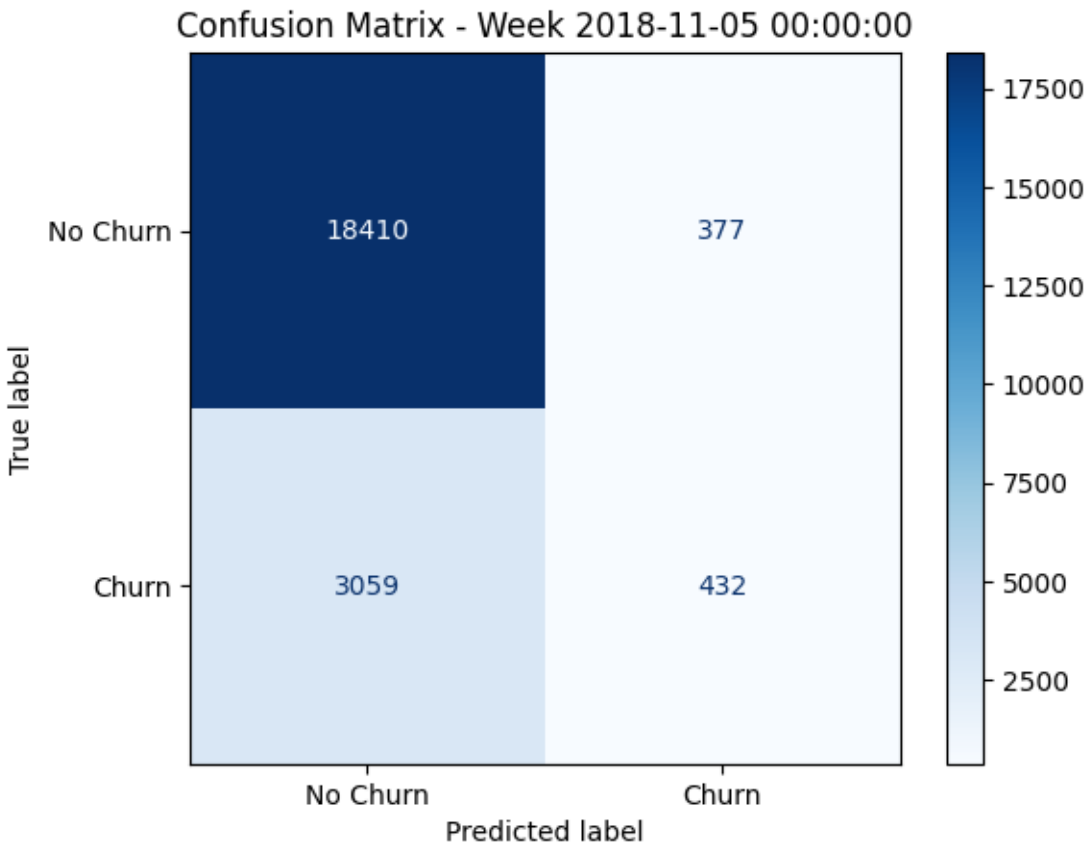
Week 2018-10-29 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.848489	0.979026	0.909095	18642.000000	0.979026
1	0.490885	0.103685	0.171208	3636.000000	NaN
accuracy	0.836161	0.836161	0.836161	0.836161	0.836161
macro avg	0.669687	0.541356	0.540152	22278.000000	NaN
weighted avg	0.790124	0.836161	0.788665	22278.000000	NaN



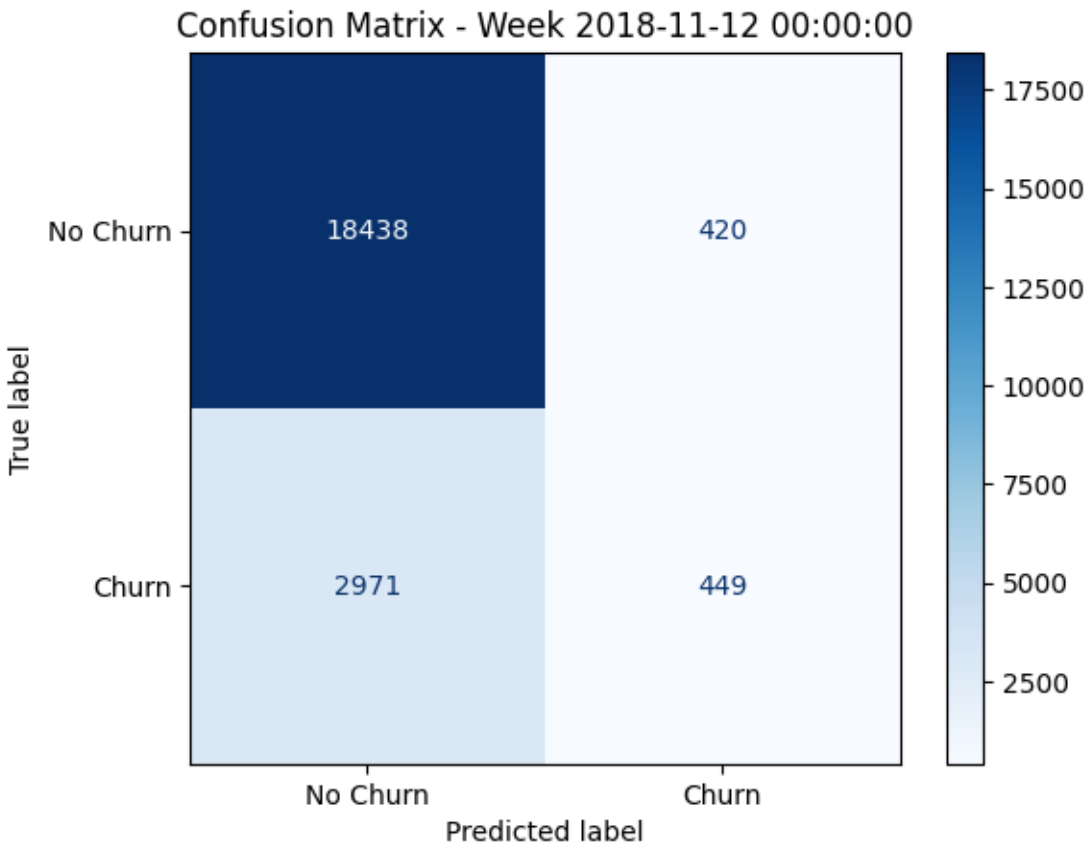
Week 2018-11-05 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.857515	0.979933	0.914646	18787.000000	0.979933
1	0.533993	0.123747	0.200930	3491.000000	NaN
accuracy	0.845767	0.845767	0.845767	0.845767	0.845767
macro avg	0.695754	0.551840	0.557788	22278.000000	NaN
weighted avg	0.806819	0.845767	0.802806	22278.000000	NaN



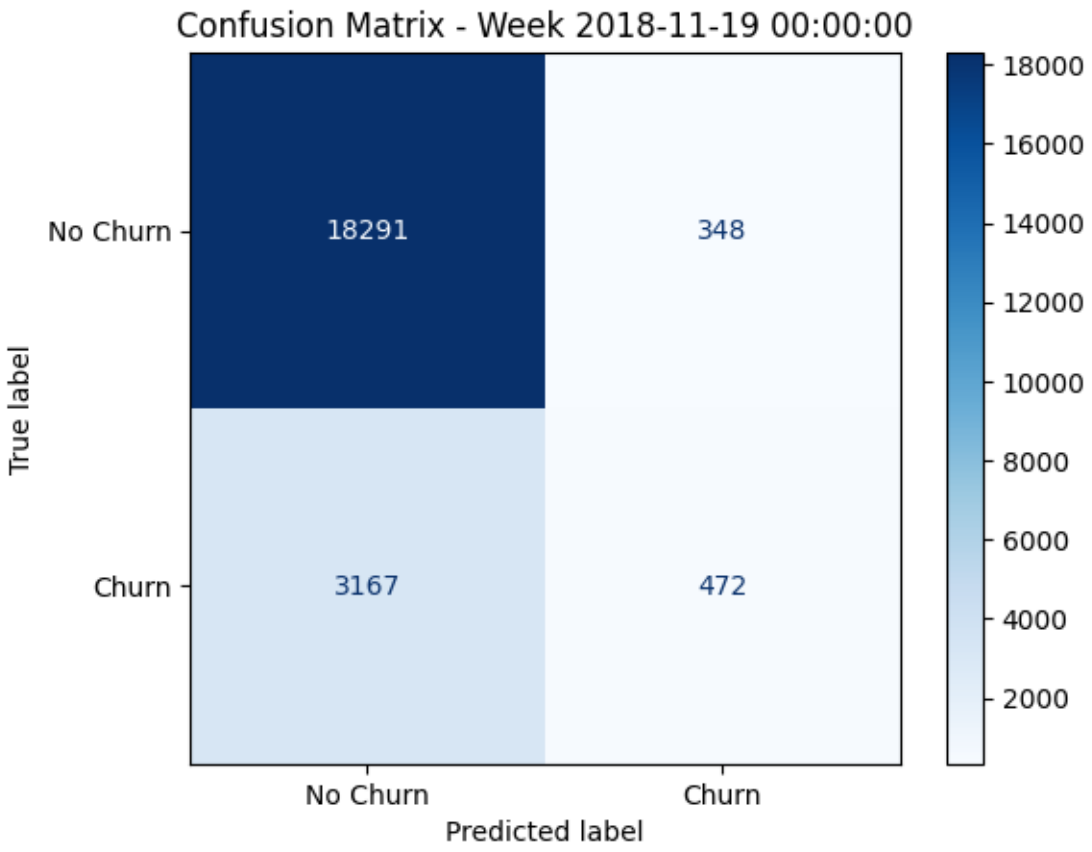
Week 2018-11-12 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.861227	0.977728	0.915787	18858.000000	0.977728
1	0.516686	0.131287	0.209373	3420.000000	NaN
accuracy	0.847787	0.847787	0.847787	0.847787	0.847787
macro avg	0.688956	0.554507	0.562580	22278.000000	NaN
weighted avg	0.808335	0.847787	0.807342	22278.000000	NaN



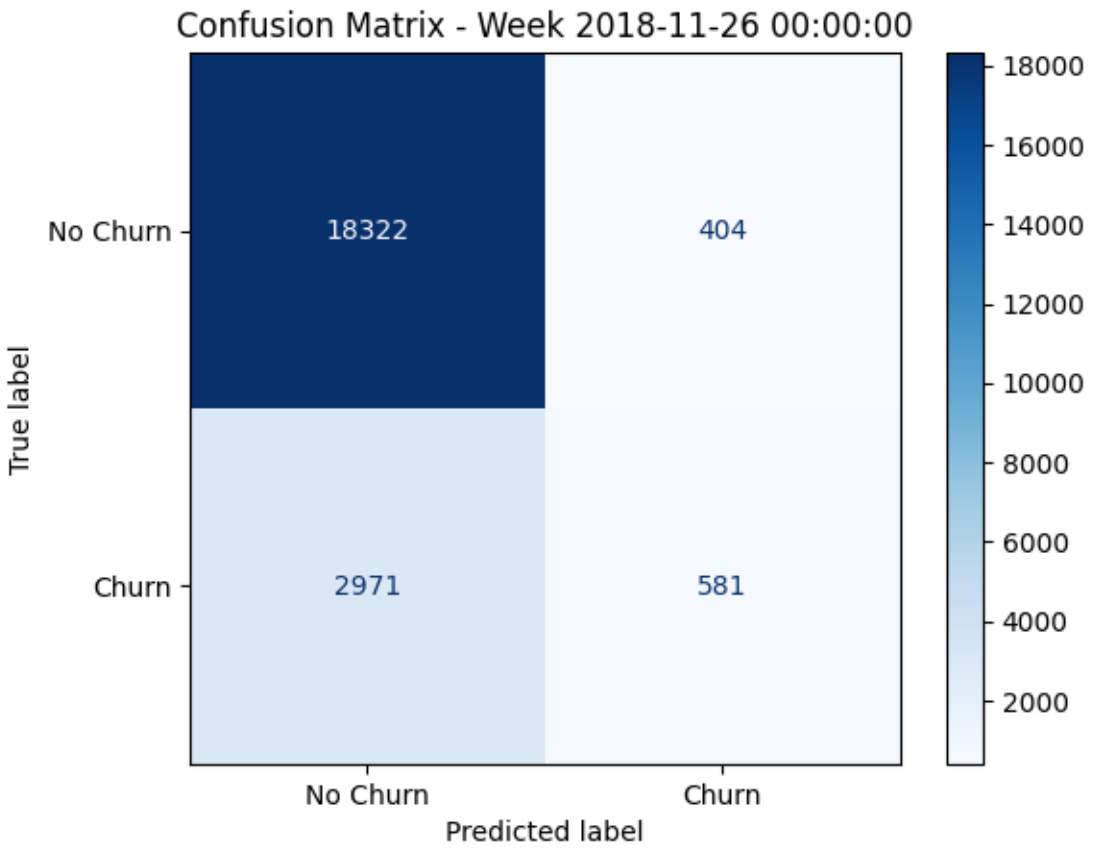
Week 2018-11-19 00:00:00:

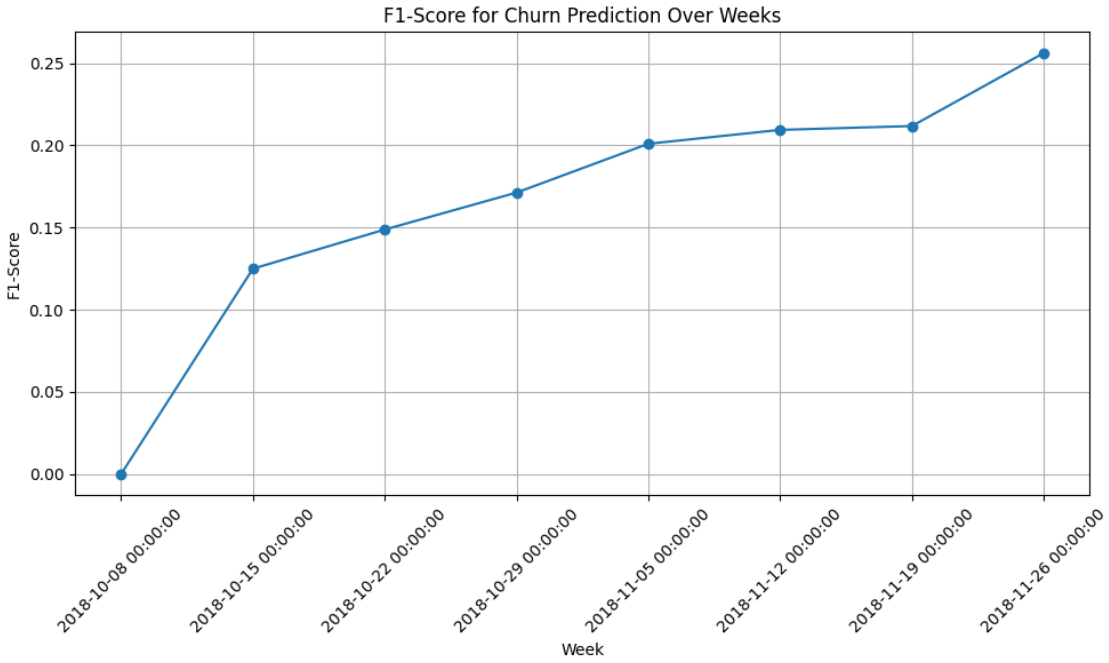
	precision	recall	f1-score	support	specificity
0	0.852409	0.981329	0.912338	18639.000000	0.981329
1	0.575610	0.129706	0.211707	3639.000000	NaN
accuracy	0.842221	0.842221	0.842221	0.842221	0.842221
macro avg	0.714010	0.555518	0.562022	22278.000000	NaN
weighted avg	0.807196	0.842221	0.797893	22278.000000	NaN



Week 2018-11-26 00:00:00:

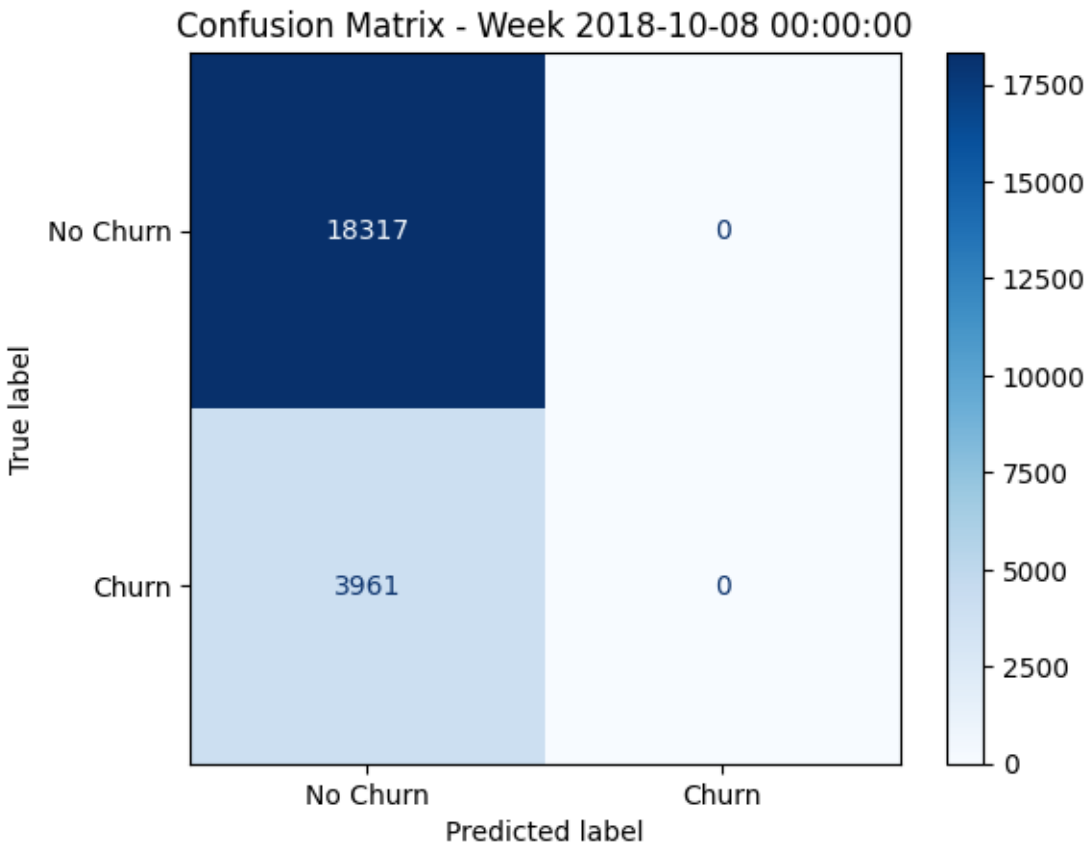
	precision	recall	f1-score	support	specificity
0	0.860471	0.978426	0.915665	18726.000000	0.978426
1	0.589848	0.163570	0.256116	3552.000000	NaN
accuracy	0.848505	0.848505	0.848505	0.848505	0.848505
macro avg	0.725159	0.570998	0.585891	22278.000000	NaN
weighted avg	0.817323	0.848505	0.810507	22278.000000	NaN


Confusion Matrix - Week 2018-11-26 00:00:00
No Churn
Churn
True label
18322
404
2971
581
No Churn
Churn
Predicted label
18000
16000
14000
12000
10000
8000
6000
4000
2000


F1-Score for Churn Prediction Over Weeks
F1-Score
0.25
0.20
0.15
0.10
0.05
0.00
2018-10-08 00:00:00
2018-10-15 00:00:00
2018-10-22 00:00:00
2018-10-29 00:00:00
2018-11-05 00:00:00
2018-11-12 00:00:00
2018-11-19 00:00:00
2018-11-26 00:00:00
Week

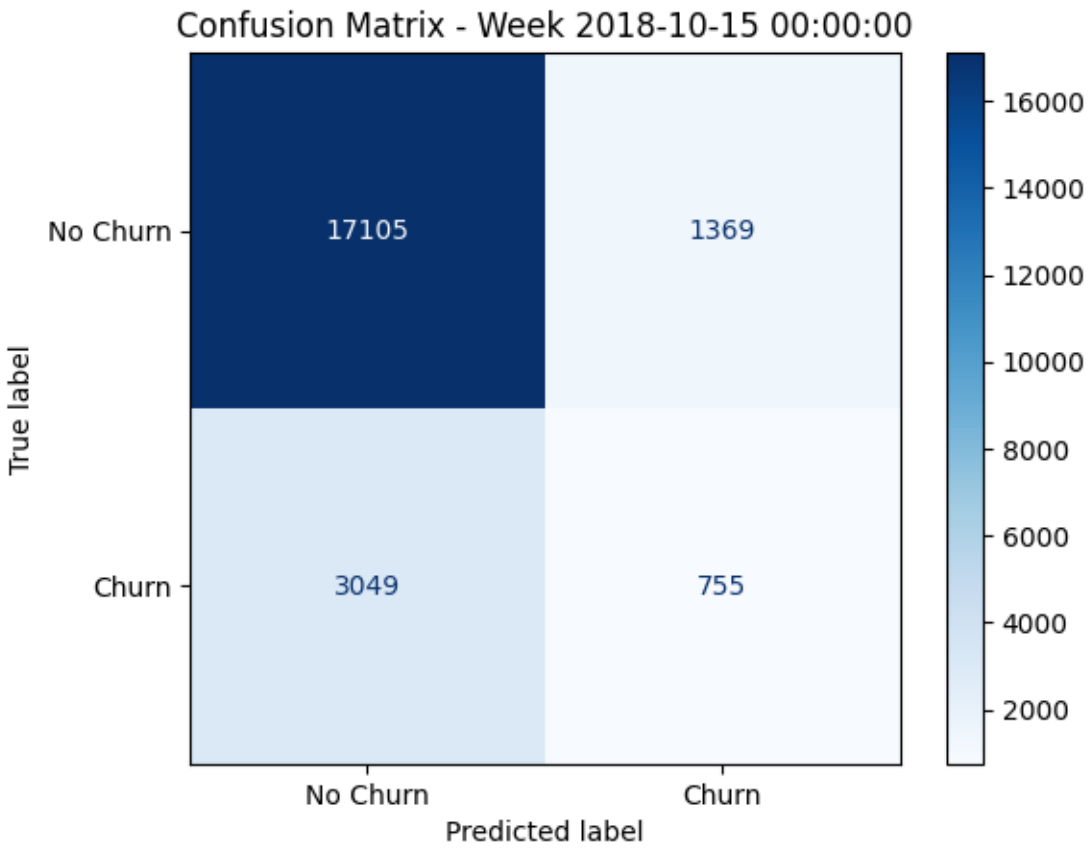
[250]:
```
# Fitting a Decision Tree
model = DecisionTreeClassifier(random_state=42, class_weight='balanced')
features = ['prev_week_activity',
            'prev_week_session_duration',
            'prev_dummy_level',
            'rolling_diff_1_weeks',
            'rolling_diff_2_weeks',
            'rolling_diff_3_weeks',
            'rolling_diff_4_weeks',
            'rolling_diff_5_weeks',
            'rolling_diff_6_weeks',
            'rolling_diff_7_weeks',
            'rolling_diff_8_weeks',
            'prev_week_rolling_diff_7_days',
            'prev_week_rolling_diff_14_days',
            'prev_week_rolling_diff_21_days',
            'prev_week_rolling_diff_28_days'
            ]
label = 'churn'

rolling_churn_prediction(df = df_all_weeks, features=features, label=label,␣
 ↪model=model)
```

```
Model type: <class 'sklearn.tree._classes.DecisionTreeClassifier'>
Week 2018-10-08 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.822201  1.000000  0.902426  18317.000000     1.000000
1              0.000000  0.000000  0.000000   3961.000000          NaN
accuracy       0.822201  0.822201  0.822201      0.822201     0.822201
macro avg      0.411101  0.500000  0.451213  22278.000000          NaN
weighted avg   0.676015  0.822201  0.741976  22278.000000          NaN
```



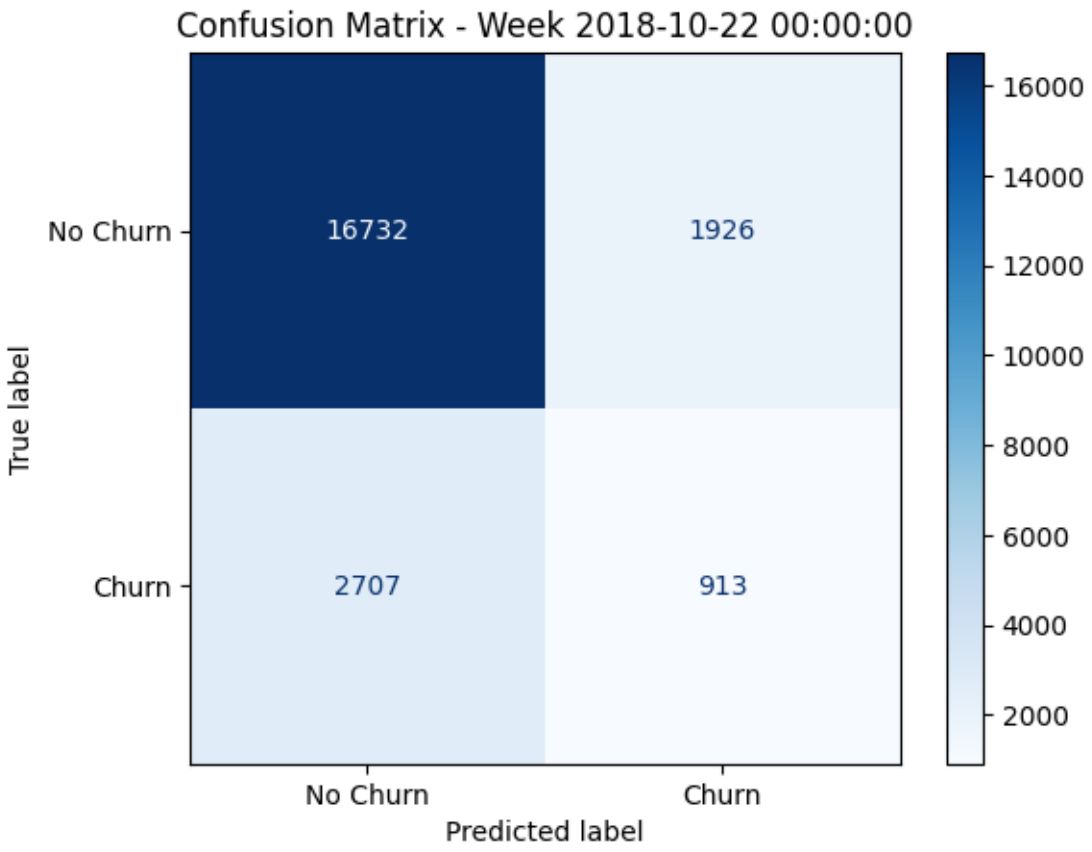
Week 2018-10-15 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.848715	0.925896	0.885627	18474.000000	0.925896
1	0.355461	0.198475	0.254723	3804.000000	NaN
accuracy	0.801688	0.801688	0.801688	0.801688	0.801688
macro avg	0.602088	0.562186	0.570175	22278.000000	NaN
weighted avg	0.764491	0.801688	0.777899	22278.000000	NaN



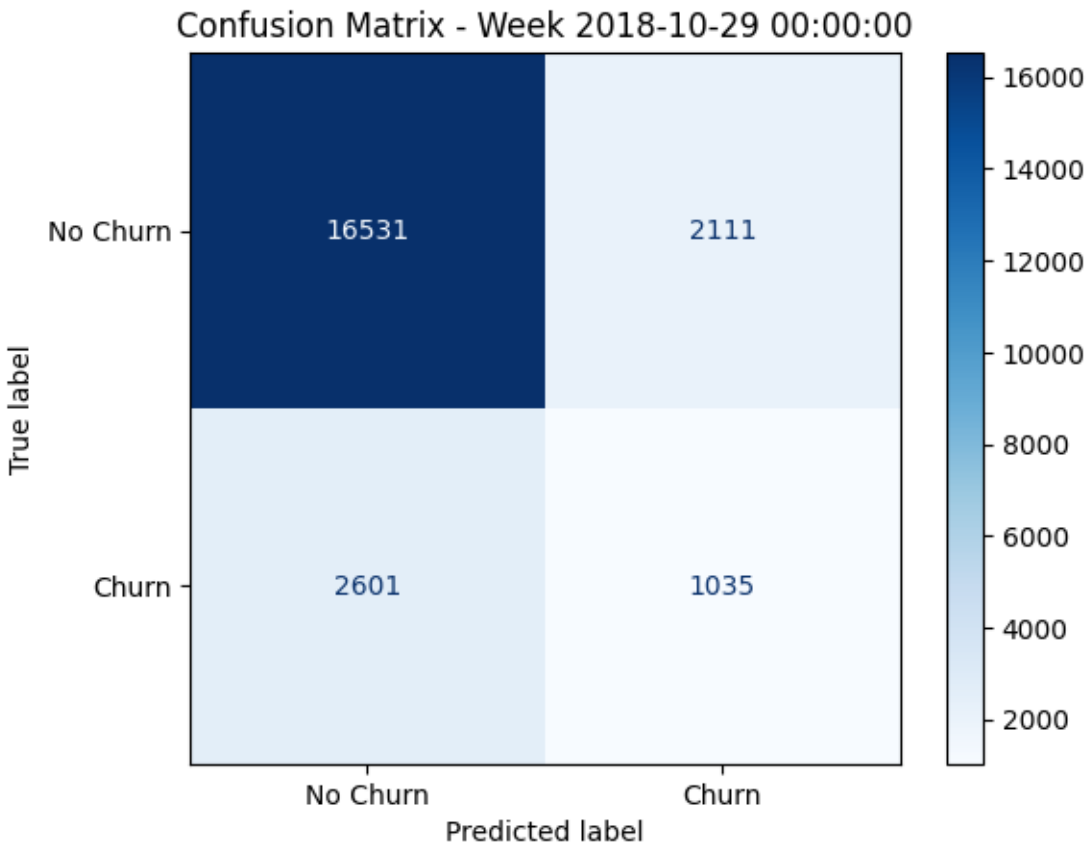
Week 2018-10-22 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.860744	0.896774	0.878389	18658.000000	0.896774
1	0.321592	0.252210	0.282706	3620.000000	NaN
accuracy	0.792037	0.792037	0.792037	0.792037	0.792037
macro avg	0.591168	0.574492	0.580548	22278.000000	NaN
weighted avg	0.773136	0.792037	0.781596	22278.000000	NaN

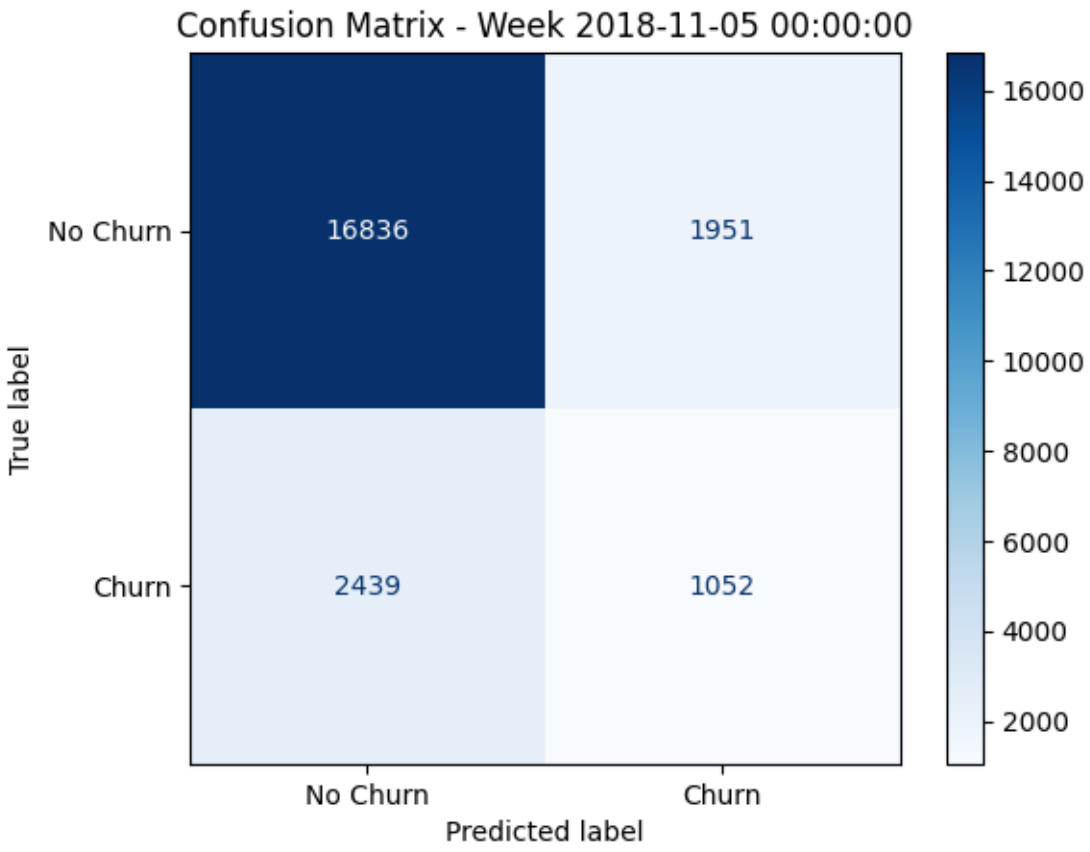


Week 2018-10-29 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.864050	0.886761	0.875258	18642.000000	0.886761
1	0.328989	0.284653	0.305220	3636.000000	NaN
accuracy	0.788491	0.788491	0.788491	0.788491	0.788491
macro avg	0.596519	0.585707	0.590239	22278.000000	NaN
weighted avg	0.776722	0.788491	0.782222	22278.000000	NaN

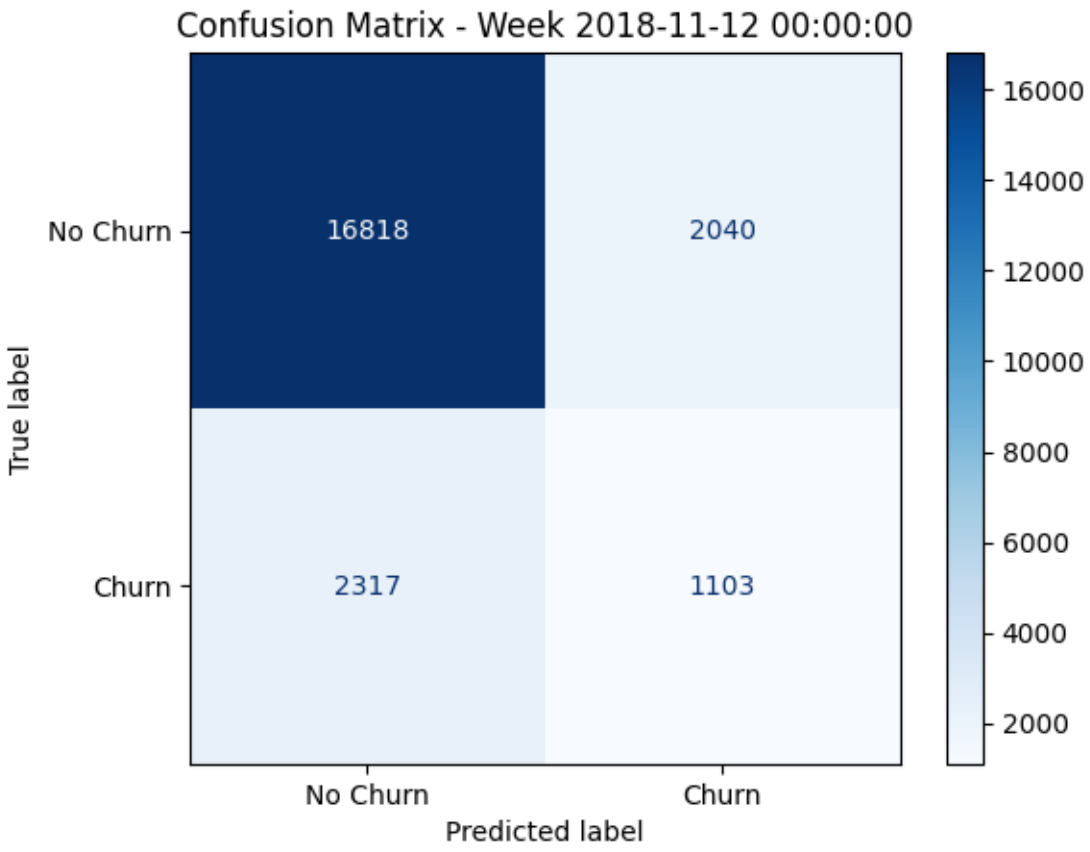



```
--------------------------------------------------
Week 2018-11-05 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.873463  0.896152  0.884662  18787.000000     0.896152
1              0.350316  0.301346  0.323991   3491.000000          NaN
accuracy       0.802945  0.802945  0.802945      0.802945     0.802945
macro avg      0.611890  0.598749  0.604327  22278.000000          NaN
weighted avg   0.791485  0.802945  0.796804  22278.000000          NaN
```



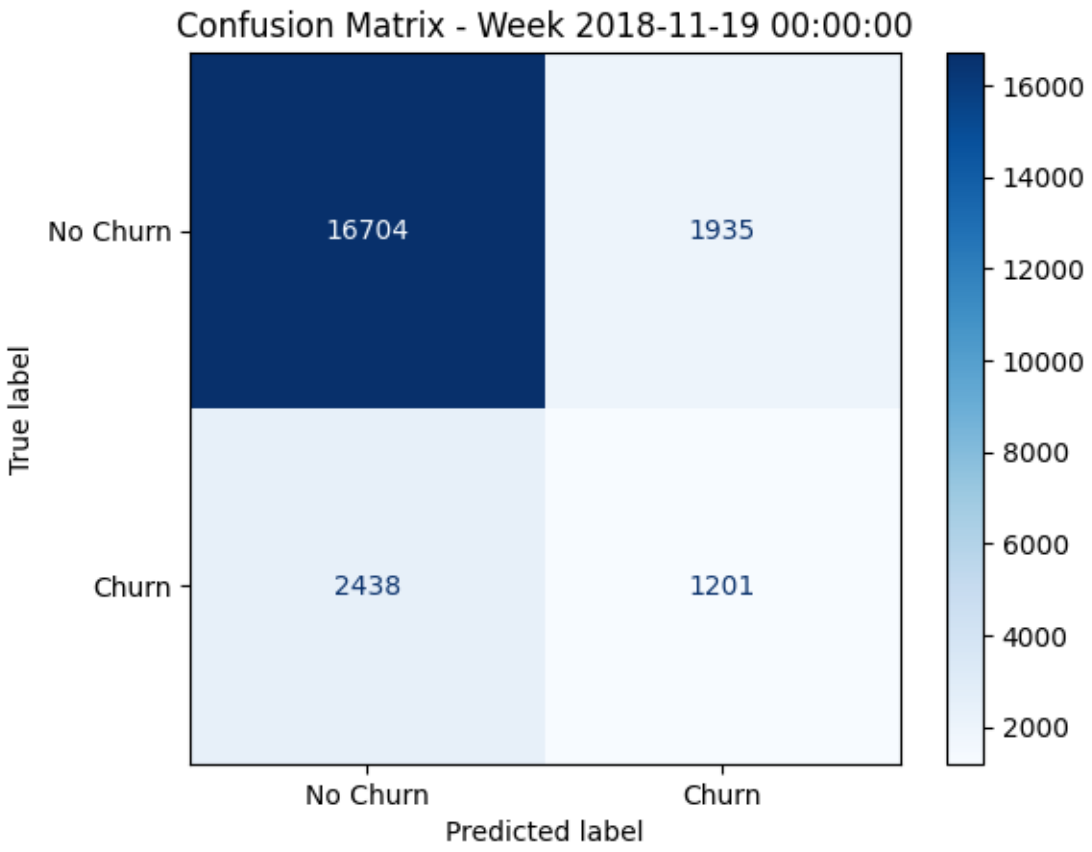
Week 2018-11-12 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.878913	0.891823	0.885321	18858.000000	0.891823
1	0.350939	0.322515	0.336127	3420.000000	NaN
accuracy	0.804426	0.804426	0.804426	0.804426	0.804426
macro avg	0.614926	0.607169	0.610724	22278.000000	NaN
weighted avg	0.797861	0.804426	0.801012	22278.000000	NaN



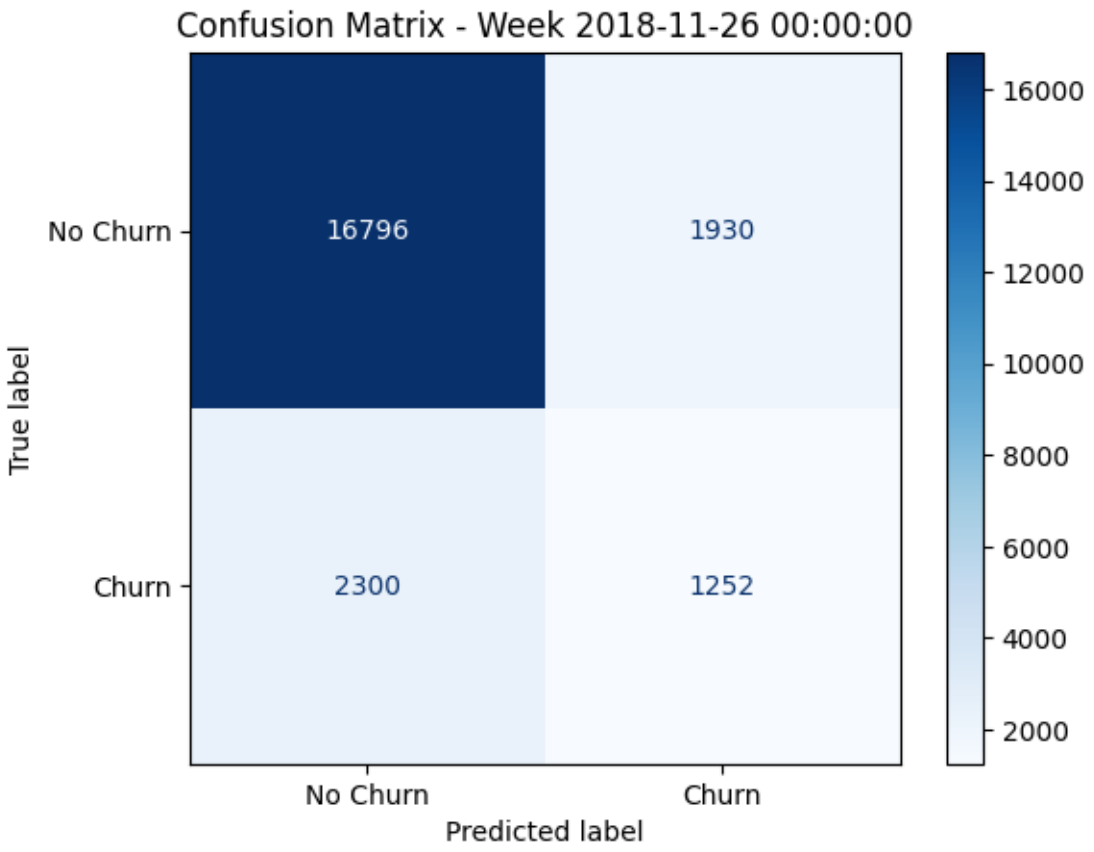
Week 2018-11-19 00:00:00:

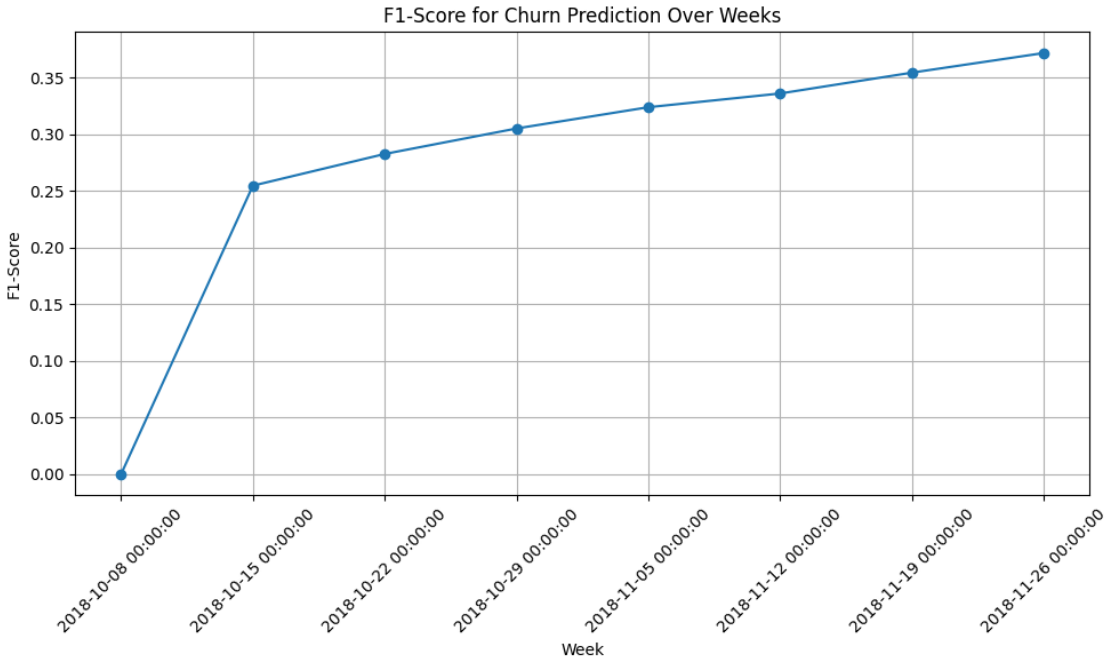
	precision	recall	f1-score	support	specificity
0	0.872636	0.896185	0.884254	18639.000000	0.896185
1	0.382972	0.330036	0.354539	3639.000000	NaN
accuracy	0.803708	0.803708	0.803708	0.803708	0.803708
macro avg	0.627804	0.613111	0.619396	22278.000000	NaN
weighted avg	0.792652	0.803708	0.797728	22278.000000	NaN



Week 2018-11-26 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.879556	0.896935	0.888160	18726.000000	0.896935
1	0.393463	0.352477	0.371844	3552.000000	NaN
accuracy	0.810127	0.810127	0.810127	0.810127	0.810127
macro avg	0.636510	0.624706	0.630002	22278.000000	NaN
weighted avg	0.802053	0.810127	0.805839	22278.000000	NaN


Confusion Matrix - Week 2018-11-26 00:00:00
16796
1930
2300
1252
No Churn
Churn
True label
No Churn
Churn
Predicted label
16000
14000
12000
10000
8000
6000
4000
2000


F1-Score for Churn Prediction Over Weeks
F1-Score
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
2018-10-08 00:00:00
2018-10-15 00:00:00
2018-10-22 00:00:00
2018-10-29 00:00:00
2018-11-05 00:00:00
2018-11-12 00:00:00
2018-11-19 00:00:00
2018-11-26 00:00:00
Week

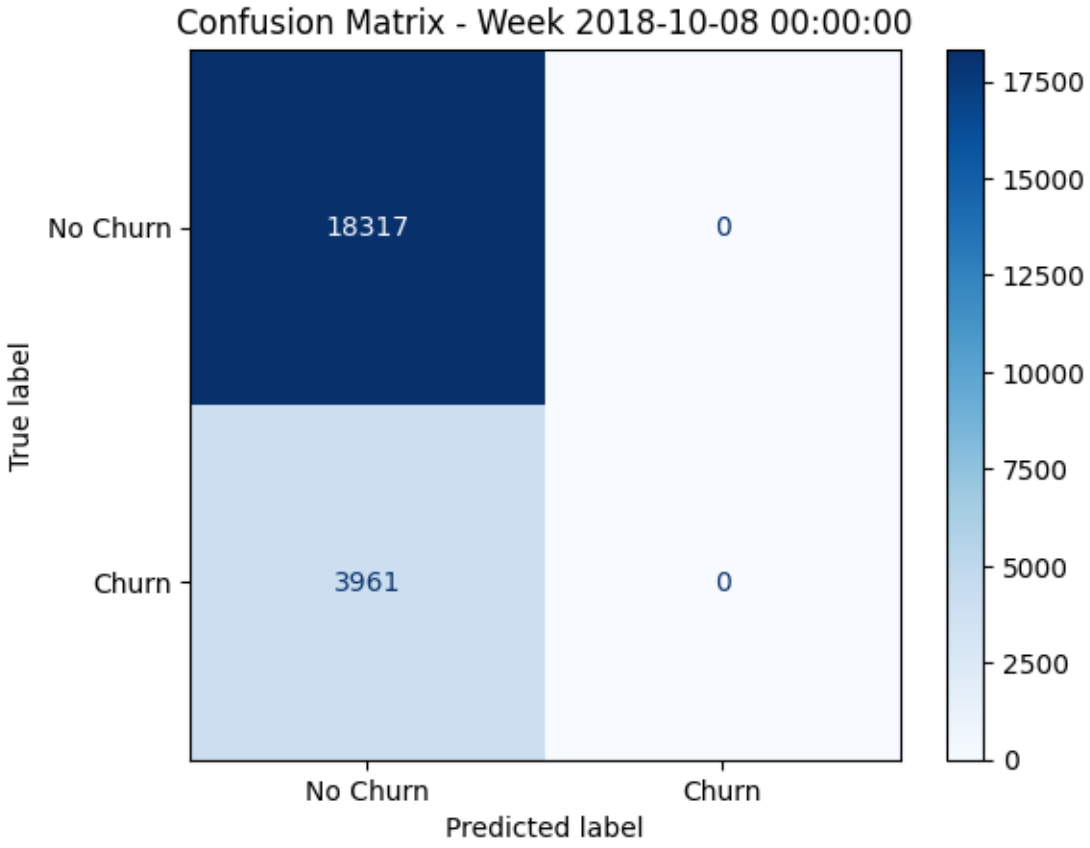
[ ]:
```
print(df_all_weeks[features + [label]].corr()[label].
  ↪sort_values(ascending=False))
```

```
churn                          1.000000
prev_week_session_duration     0.273998
rolling_diff_4_weeks           0.156740
rolling_diff_5_weeks           0.155946
rolling_diff_3_weeks           0.155399
rolling_diff_2_weeks           0.153361
rolling_diff_6_weeks           0.152086
rolling_diff_7_weeks           0.149336
rolling_diff_8_weeks           0.147695
prev_week_activity             0.142876
rolling_diff_1_weeks           0.138172
prev_dummy_level              -0.037114
Name: churn, dtype: float64
```

[251]:
```
model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500,␣
  ↪random_state=42)
features = ['prev_week_activity',
            'prev_week_session_duration',
            'prev_dummy_level',
            'rolling_diff_1_weeks',
            'rolling_diff_2_weeks',
            'rolling_diff_3_weeks',
            'rolling_diff_4_weeks',
            'rolling_diff_5_weeks',
            'rolling_diff_6_weeks',
            'rolling_diff_7_weeks',
            'rolling_diff_8_weeks',
            'prev_week_rolling_diff_7_days',
            'prev_week_rolling_diff_14_days',
            'prev_week_rolling_diff_21_days',
            'prev_week_rolling_diff_28_days'
           ]
label = 'churn'

rolling_churn_prediction(df = df_all_weeks, features=features, label=label,␣
  ↪model=model)
```

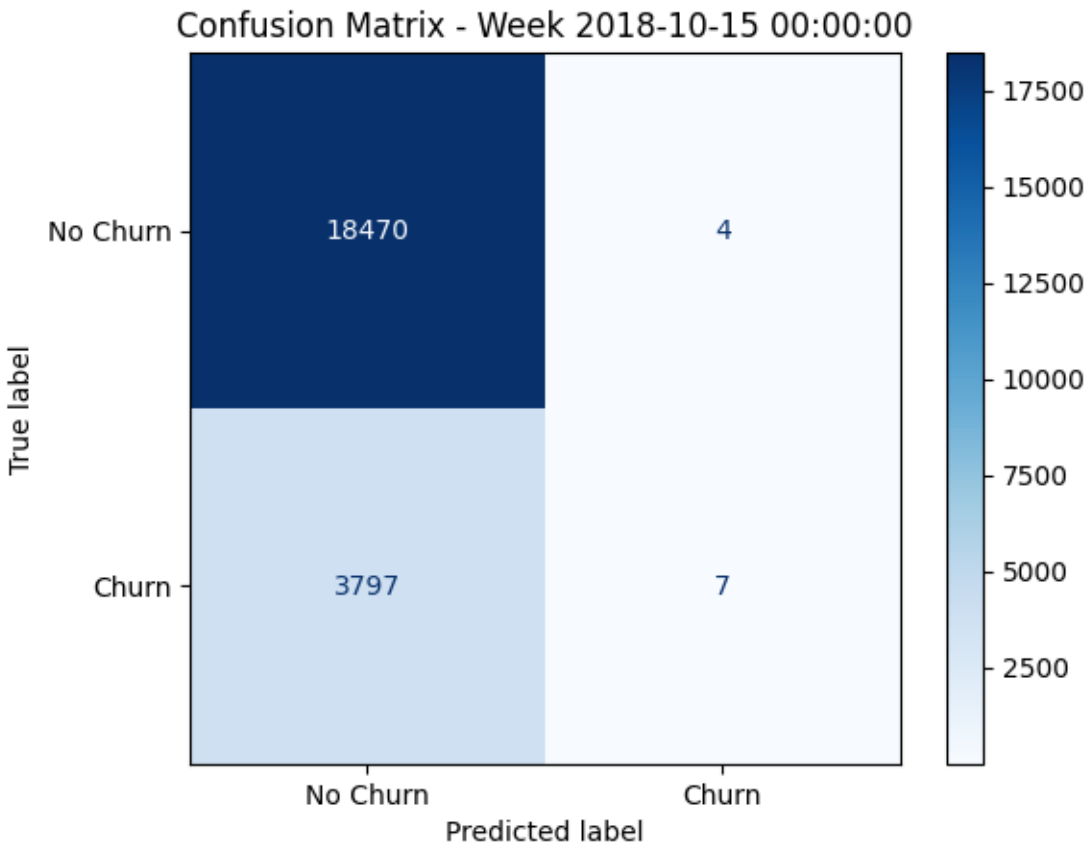
```
Model type: <class
'sklearn.neural_network._multilayer_perceptron.MLPClassifier'>
Week 2018-10-08 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.822201  1.000000  0.902426  18317.000000     1.000000
1              0.000000  0.000000  0.000000   3961.000000          NaN
```

accuracy	0.822201	0.822201	0.822201	0.822201	0.822201
macro avg	0.411101	0.500000	0.451213	22278.000000	NaN
weighted avg	0.676015	0.822201	0.741976	22278.000000	NaN

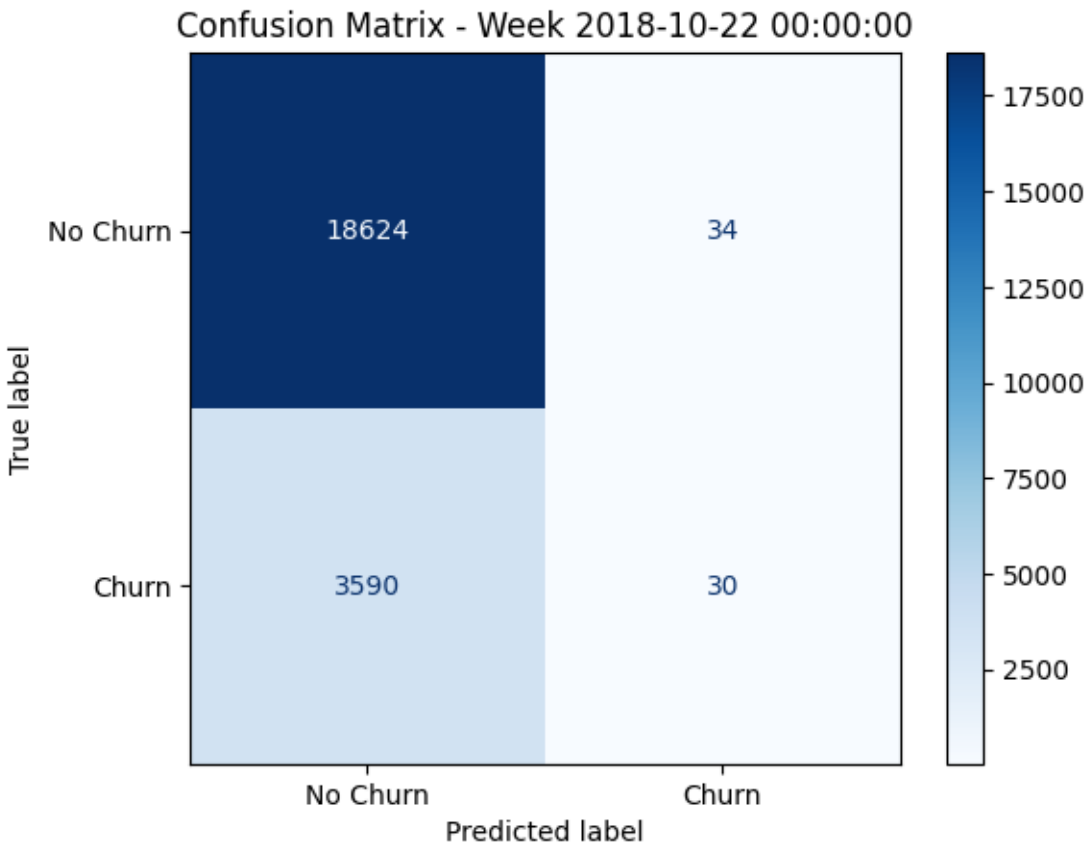


Week 2018-10-15 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.829479	0.999783	0.906703	18474.000000	0.999783
1	0.636364	0.001840	0.003670	3804.000000	NaN
accuracy	0.829383	0.829383	0.829383	0.829383	0.829383
macro avg	0.732921	0.500812	0.455187	22278.000000	NaN
weighted avg	0.796504	0.829383	0.752509	22278.000000	NaN

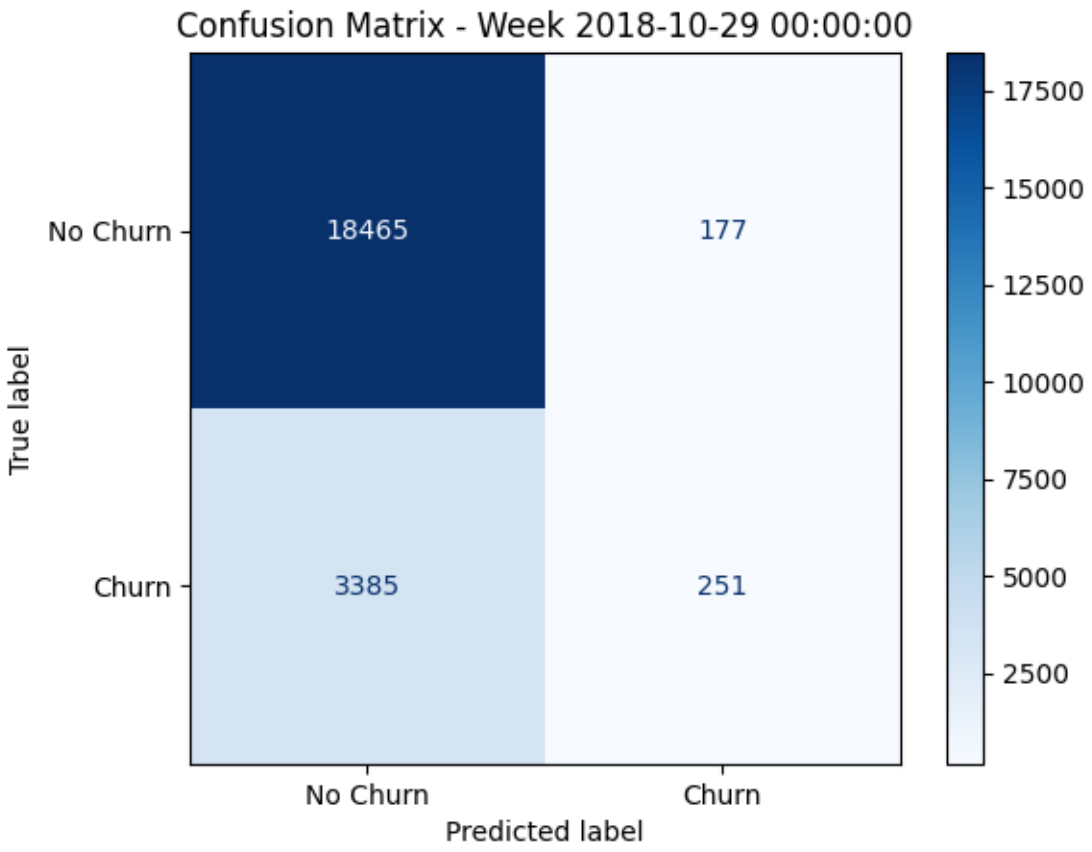



```
--------------------------------------------------
Week 2018-10-22 00:00:00:
              precision    recall  f1-score       support  specificity
0              0.838390  0.998178  0.911333  18658.000000     0.998178
1              0.468750  0.008287  0.016287   3620.000000          NaN
accuracy       0.837328  0.837328  0.837328      0.837328     0.837328
macro avg      0.653570  0.503233  0.463810  22278.000000          NaN
weighted avg   0.778327  0.837328  0.765895  22278.000000          NaN
```



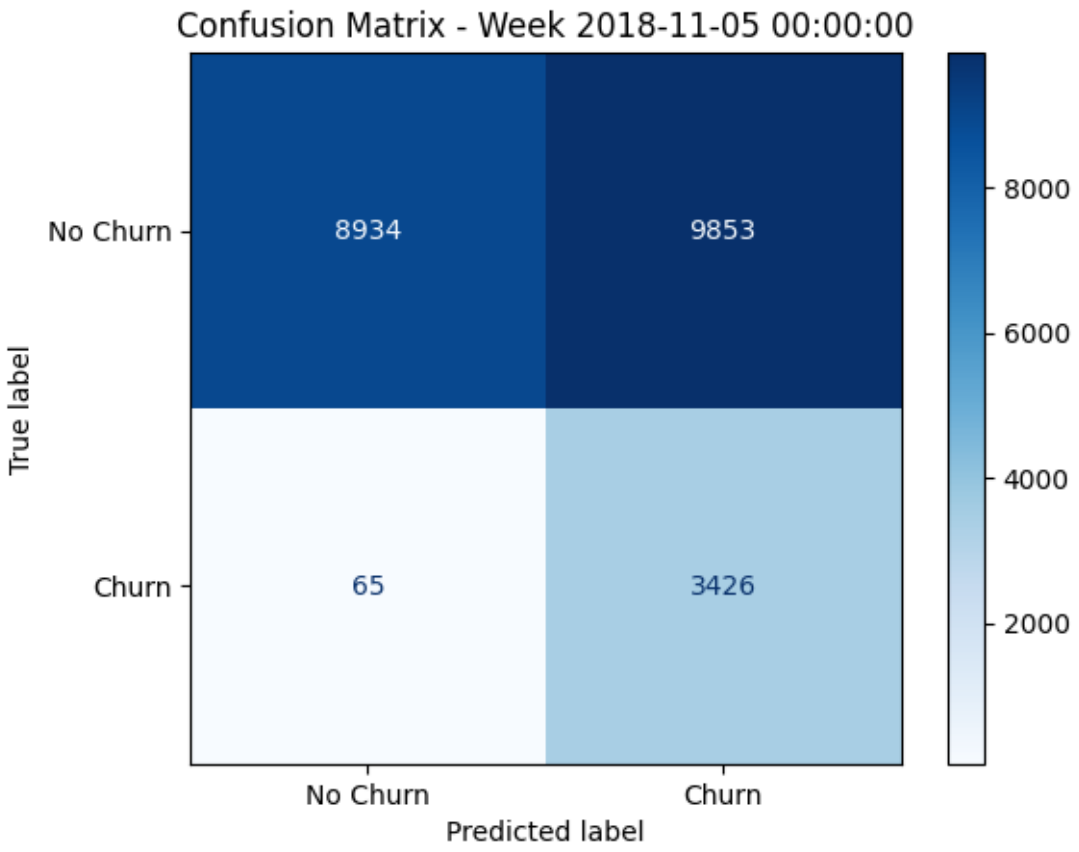
Week 2018-10-29 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.845080	0.990505	0.912032	18642.000000	0.990505
1	0.586449	0.069032	0.123524	3636.000000	NaN
accuracy	0.840111	0.840111	0.840111	0.840111	0.840111
macro avg	0.715764	0.529769	0.517778	22278.000000	NaN
weighted avg	0.802869	0.840111	0.783339	22278.000000	NaN



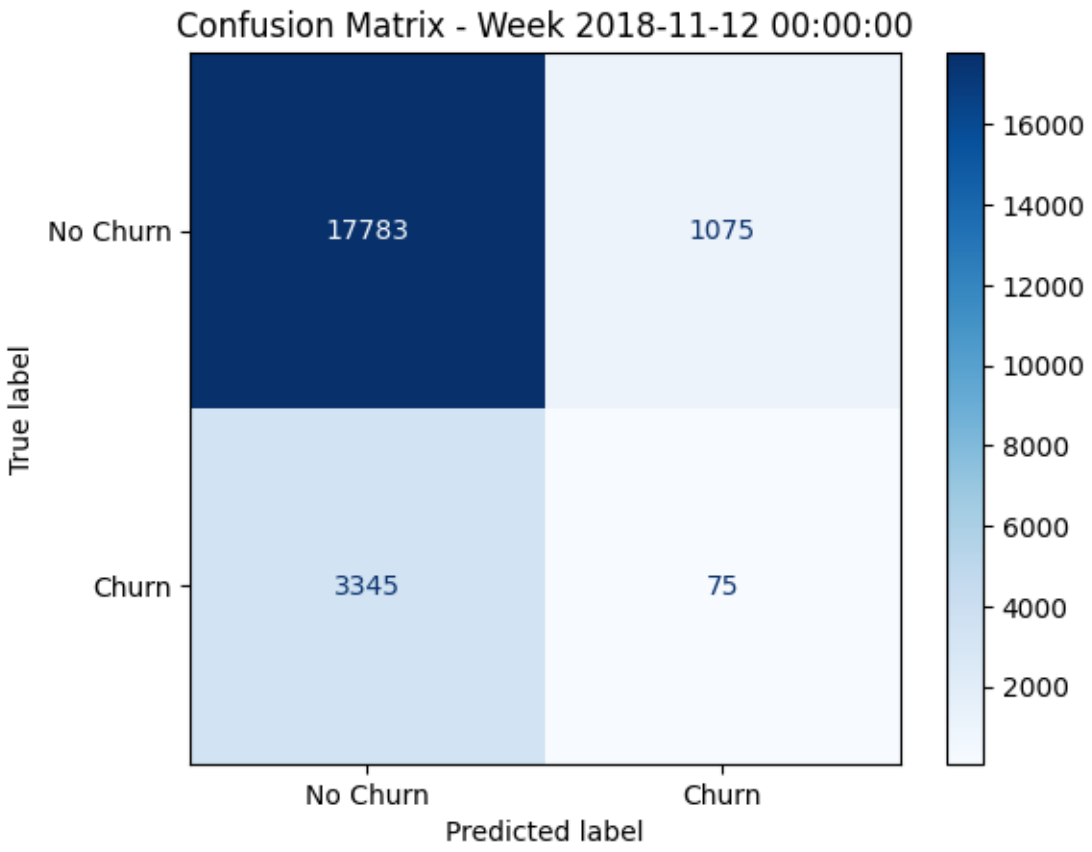
Week 2018-11-05 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.992777	0.475542	0.643058	18787.000000	0.475542
1	0.258001	0.981381	0.408587	3491.000000	NaN
accuracy	0.554807	0.554807	0.554807	0.554807	0.554807
macro avg	0.625389	0.728461	0.525822	22278.000000	NaN
weighted avg	0.877636	0.554807	0.606316	22278.000000	NaN



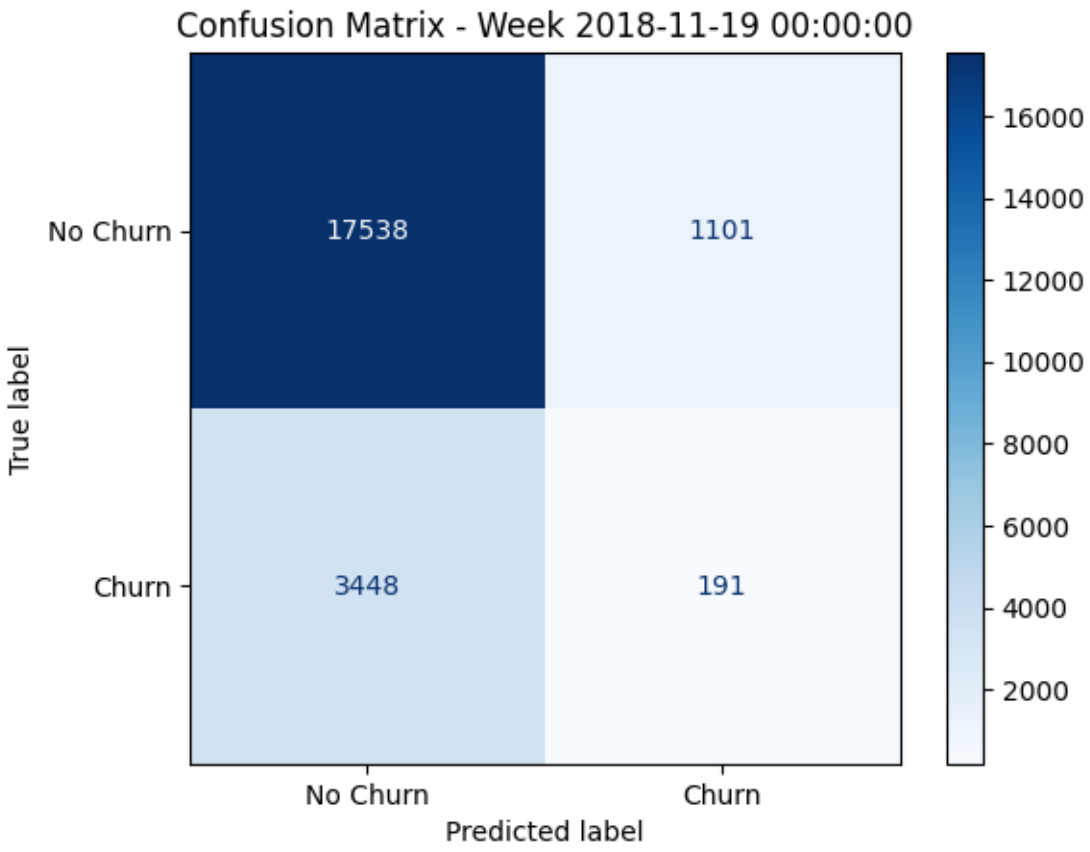
Week 2018-11-12 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.841679	0.942995	0.889461	18858.000000	0.942995
1	0.065217	0.021930	0.032823	3420.000000	NaN
accuracy	0.801598	0.801598	0.801598	0.801598	0.801598
macro avg	0.453448	0.482462	0.461142	22278.000000	NaN
weighted avg	0.722481	0.801598	0.757955	22278.000000	NaN



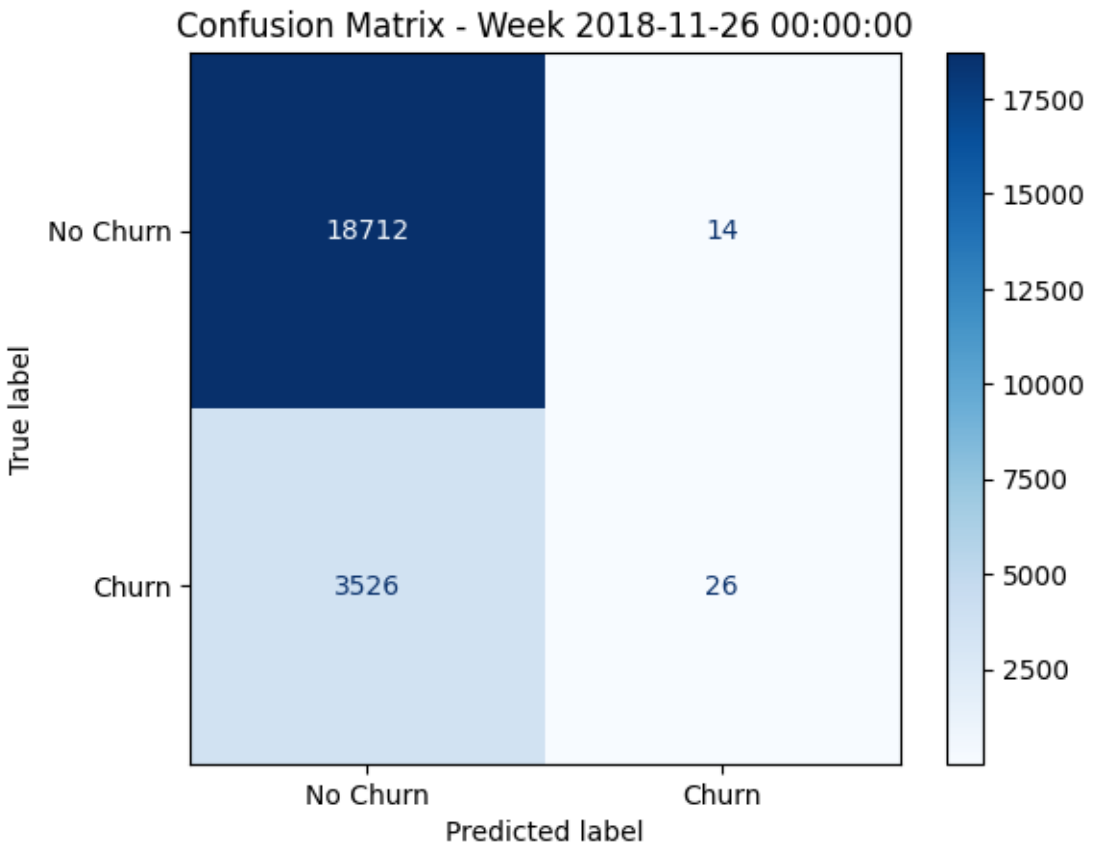
Week 2018-11-19 00:00:00:

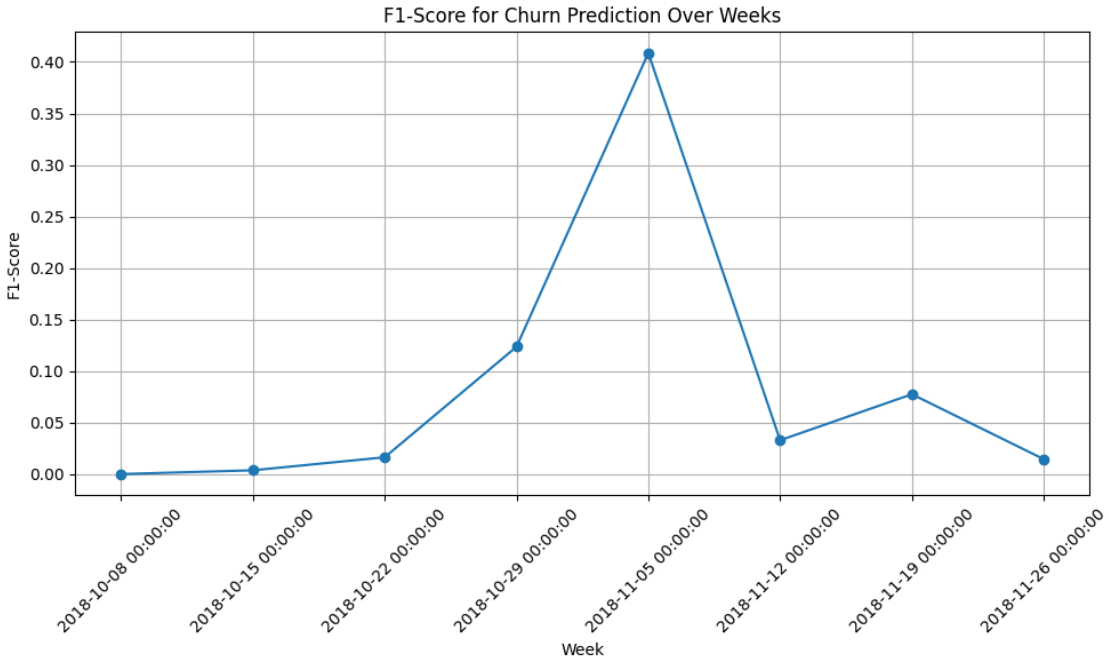
	precision	recall	f1-score	support	specificity
0	0.835700	0.940930	0.885199	18639.000000	0.940930
1	0.147833	0.052487	0.077469	3639.000000	NaN
accuracy	0.795808	0.795808	0.795808	0.795808	0.795808
macro avg	0.491766	0.496709	0.481334	22278.000000	NaN
weighted avg	0.723340	0.795808	0.753260	22278.000000	NaN



Week 2018-11-26 00:00:00:

	precision	recall	f1-score	support	specificity
0	0.841443	0.999252	0.913583	18726.000000	0.999252
1	0.650000	0.007320	0.014477	3552.000000	NaN
accuracy	0.841099	0.841099	0.841099	0.841099	0.841099
macro avg	0.745721	0.503286	0.464030	22278.000000	NaN
weighted avg	0.810919	0.841099	0.770229	22278.000000	NaN


Confusion Matrix - Week 2018-11-26 00:00:00
No Churn
Churn
True label
18712
14
3526
26
No Churn
Churn
Predicted label
17500
15000
12500
10000
7500
5000
2500


F1-Score for Churn Prediction Over Weeks
F1-Score
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
2018-10-08 00:00:00
2018-10-15 00:00:00
2018-10-22 00:00:00
2018-10-29 00:00:00
2018-11-05 00:00:00
2018-11-12 00:00:00
2018-11-19 00:00:00
2018-11-26 00:00:00
Week